
Лекция 8: Интеграция обучения с подкреплением и планирования

Содержание

- 01 Мотивация и современные применения
- 02 Классический Монте-Карло поиск по дереву
- 03 Нейронный MCTS в AlphaZero
- 04 Нейронный MCTS в MuZero
- 05 Применение MCTS при инференсе языковых моделей
- 06 Подход Dreamer

01

Мотивация и современные применения

Which tasks can
Monte Carlo tree
search solve today?



Chess, Shogi, Go

- Crazy Stone – 2006
- AlphaGo – 2015
- Alpha Zero – 2017

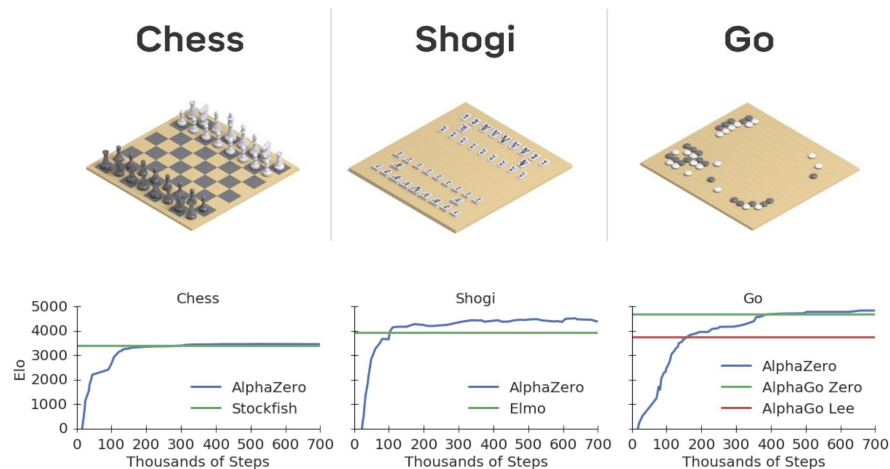


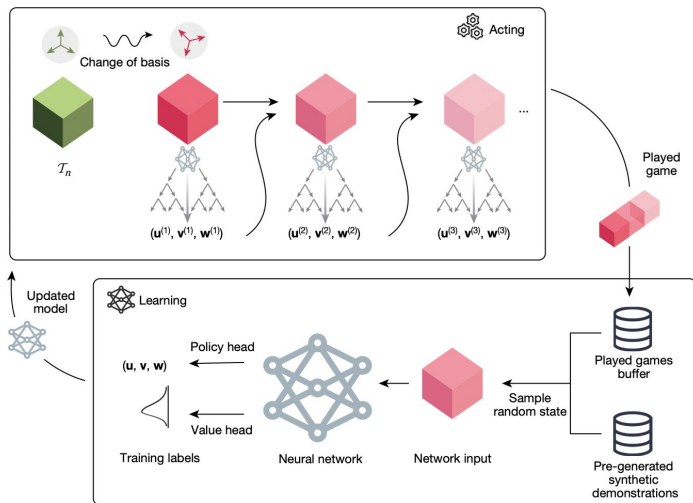
Figure 1: Training *AlphaZero* for 700,000 steps. Elo ratings were computed from evaluation games between different players when given one second per move. **a** Performance of *AlphaZero* in chess, compared to 2016 TCEC world-champion program *Stockfish*. **b** Performance of *AlphaZero* in shogi, compared to 2017 CSA world-champion program *Elmo*. **c** Performance of *AlphaZero* in Go, compared to *AlphaGo Lee* and *AlphaGo Zero* (20 block / 3 day) (29).

Atari 2600

→ Muzero — 2019



Applications: Efficient Matrix Multiplication



$n \times m$ with $m \times p$ matrices

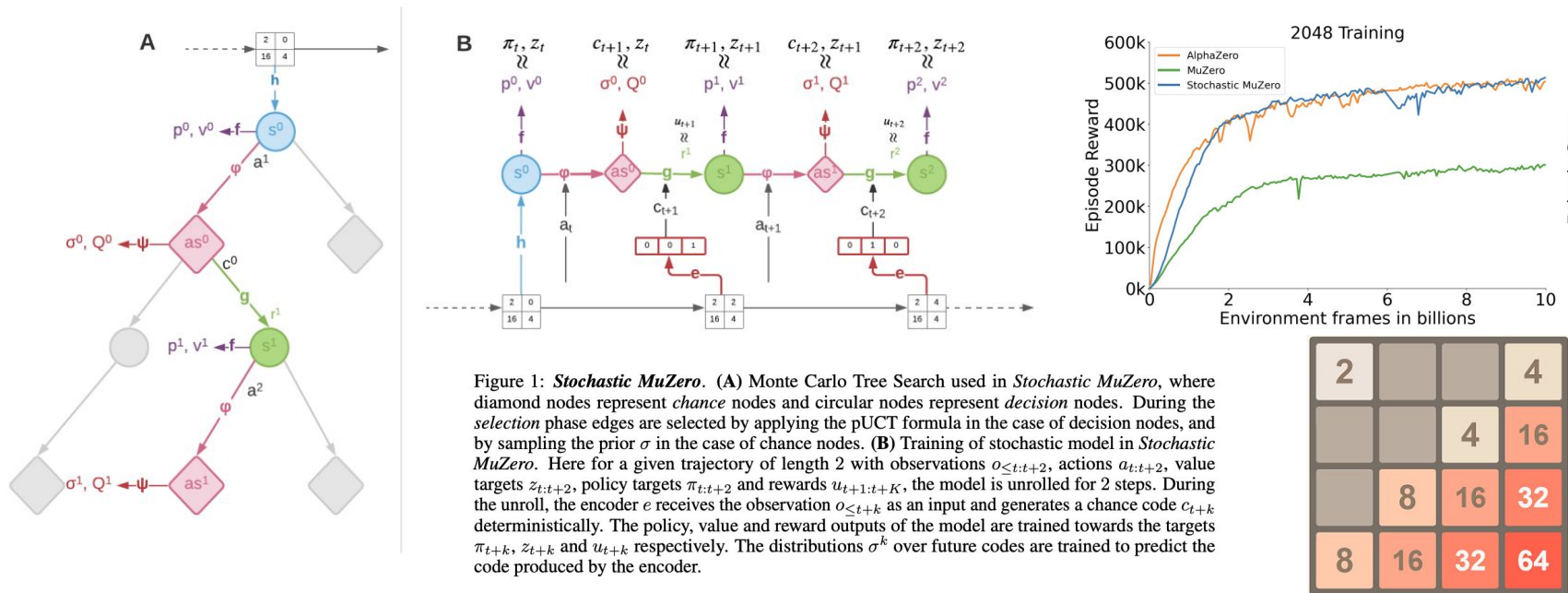
Size (n, m, p)	Best method known	Best rank known	AlphaTensor rank	Modular Standard
(2, 2, 2)	(Strassen, 1969) ²	7	7	7
(3, 3, 3)	(Laderman, 1976) ¹⁵	23	23	23
(4, 4, 4)	(Strassen, 1969) ²	49	47	49
(5, 5, 5)	(3, 5, 5) + (2, 5, 5)	98	96	98
(2, 2, 3)	(2, 2, 2) + (2, 2, 1)	11	11	11
(2, 2, 4)	(2, 2, 2) + (2, 2, 2)	14	14	14
(2, 2, 5)	(2, 2, 2) + (2, 2, 3)	18	18	18
(2, 3, 3)	(Hopcroft and Kerr, 1971) ¹⁶	15	15	15
(2, 3, 4)	(Hopcroft and Kerr, 1971) ¹⁶	20	20	20
(2, 3, 5)	(Hopcroft and Kerr, 1971) ¹⁶	25	25	25
(2, 4, 4)	(Hopcroft and Kerr, 1971) ¹⁶	26	26	26
(2, 4, 5)	(Hopcroft and Kerr, 1971) ¹⁶	33	33	33
(2, 5, 5)	(Hopcroft and Kerr, 1971) ¹⁶	40	40	40
(3, 3, 4)	(Smirnov, 2013) ¹⁸	29	29	29
(3, 3, 5)	(Smirnov, 2013) ¹⁸	36	36	36
(3, 4, 4)	(Smirnov, 2013) ¹⁸	38	38	38
(3, 4, 5)	(Smirnov, 2013) ¹⁸	48	47	47
(3, 5, 5)	(Sedoglavac and Smirnov, 2021) ¹⁹	58	58	58
(4, 4, 5)	(4, 4, 2) + (4, 4, 3)	64	63	63
(4, 5, 5)	(2, 5, 5) $\otimes$ (2, 1, 1)	80	76	76

AlphaTensor – 2019

Discovering faster matrix multiplication algorithms with reinforcement learning

Alhussein Fawzi, Matej Balog, Aja Huang, Thomas Hubert, Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Francisco J. R. Ruiz, Julian Schrittwieser, Grzegorz Swirszcz, David Silver, Demis Hassabis & Pushmeet Kohli

Applications: Stochastic Environments



Planning in Stochastic Environments with a Learned Model

Mujoco (DeepMind Control Suite)

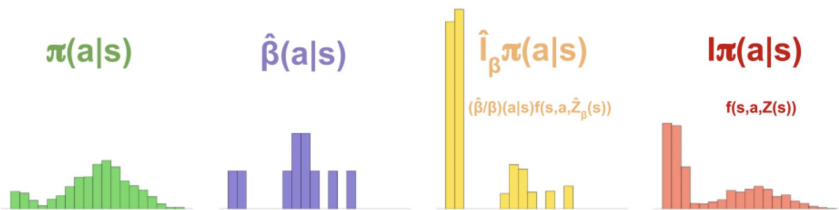
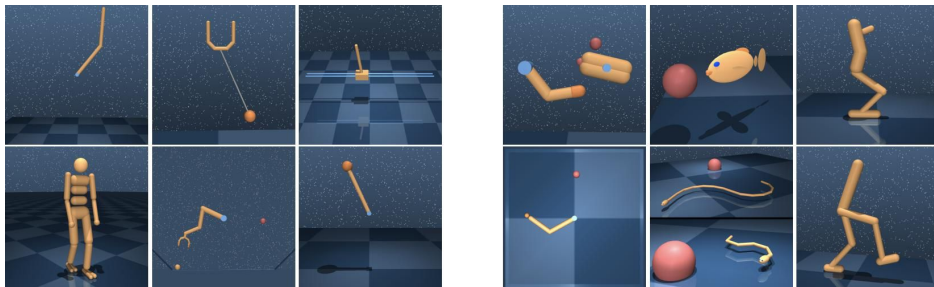


Figure 1. **Sample-based Policy Improvement.** On the left, the current policy $\pi(a|s)$. Next, K actions $\{a_i\}$ are sampled from a proposal distribution β and $\hat{\beta}(a|s)$ is the corresponding empirical distribution. A sample-based improved policy $\hat{\mathcal{I}}_{\beta}\pi(a|s) = (\hat{\beta}/\beta)(a|s)f(s,a,\hat{Z}_{\beta}(s))$ is then built. As the number of samples K increases $\hat{\mathcal{I}}_{\beta}\pi(a|s)$ converges to the improved policy $\mathcal{I}\pi(a|s) = f(s,a,Z(s))$.

Tasks	Dreamer	SMuZero
acrobot.swingup	365.26	417.52
cartpole.balance	979.56	984.86
cartpole.balance_sparse	941.84	998.14
cartpole.swingup	833.66	868.87
cartpole.swingup_sparse	812.22	846.91
cheetah.run	894.56	914.39
ball_in_cup.catch	962.48	977.38
finger.spin	498.88	986.38
finger.turn_easy	825.86	972.53
finger.turn_hard	891.38	963.07
hopper.hop	368.97	528.24
hopper.stand	923.72	926.50
pendulum.swingup	833.00	837.76
quadruped.run	888.39	923.54
quadruped.walk	931.61	933.77
reacher.easy	935.08	982.26
reacher.hard	817.05	971.53
walker.run	824.67	931.06
walker.stand	977.99	987.79
walker.walk	961.67	975.46

Applications: Offline RL

Algorithm 1 The *Reanalyse* algorithm.

```

for step  $\leftarrow 0 \dots N$  do
   $t \sim \text{random}(1 : T)$ 
   $s_t = \text{representation}(o_{1:t}, \theta)$ 
  for  $i \leftarrow 0 \dots k$  do
     $\pi_t^i, \nu_t^i = \text{improve}(\text{representation}(o_{1:t+i}, \theta), \theta)$ 
     $p_t^i, v_t^i = \text{predict}(s_t^i, \theta)$ 
     $r_t^{i+1}, s_t^{i+1} = \text{dynamics}(s_t^i, \theta)$ 
  end for
   $l = \text{loss}(h_{t:t+k}, \{r, p, v, u, \pi, \nu\}_t^{0:k}, \theta)$ 
   $\Delta\theta = \text{optimise}(l, \theta)$ 
end for

```

Game	QR-DQN	REM	CQL(H)	MZ
asterix (1%)	166.3	386.5	592.4	27220.5
breakout	7.9	11.0	61.1	251.9
pong	-13.8	-6.9	19.3	-16.2
qbert	383.6	343.4	14012.0	6953.2
sequest	672.9	499.8	779.4	4964.0
asterix (10%)	189.2	75.1	156.3	40554.0
breakout	151.2	86.7	269.3	485.8
pong	15.1	8.9	18.5	15.6
qbert	7091.3	8624.3	13855.6	16817.9
sequest	2984.8	3936.6	3674.1	8556.3

Table 3. **Low-data Atari setting.** QR-DQN (Dabney et al., 2018), REM (Agarwal et al., 2020), CQL(H) (Kumar et al., 2020) and MuZero Unplugged results when trained on only 1% (top, 2 million frames) or 10% (bottom, 20 million frames) of Atari data. MuZero Unplugged performance improves consistently when trained on more data.

Online and Offline Reinforcement Learning by Planning with a Learned Model

Hypertree proof search for neural theorem proving

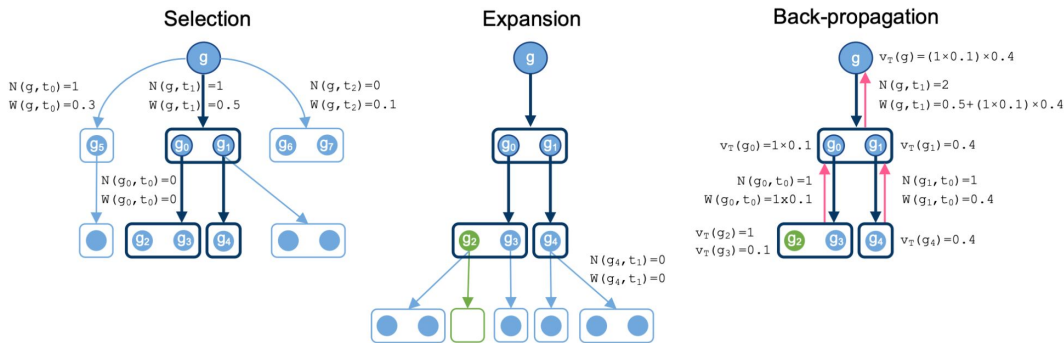
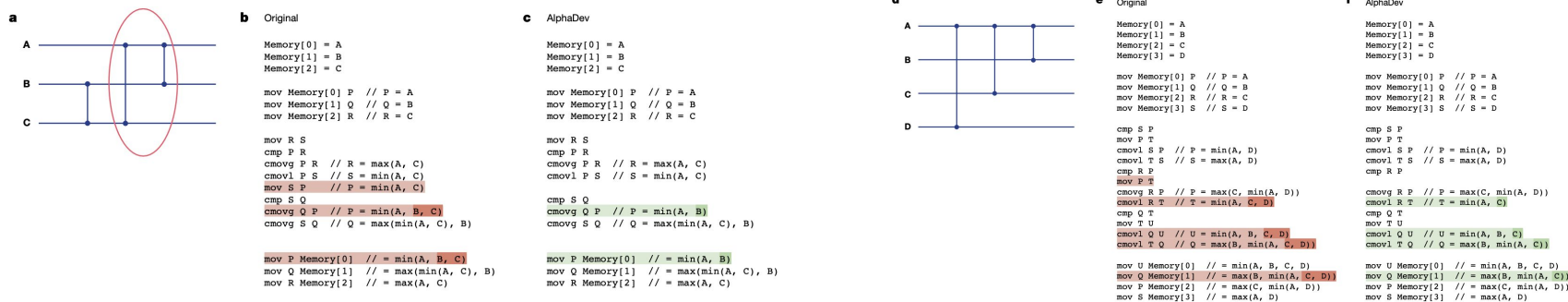


Figure 5: HyperTree Proof Search. We aim at finding a proof of the root theorem g with HTPS. Proving either $\{g_5\}$, $\{g_0, g_1\}$, or $\{g_6, g_7\}$ would lead to a proof of g by tactic t_0, t_1 , or t_2 . The figure represents the three steps of HTPS that are repeated until a proof is found. Guided by the search policy, we select a hypertree whose leaves are unexpanded nodes. The selected nodes are then expanded, adding new tactics and nodes to the hypergraph. Finally, during back-propagation we evaluate the node values of the hypertree, starting from the leaves back to the root, and update the visit counts and total action values.

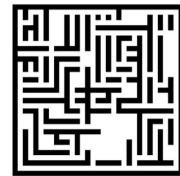
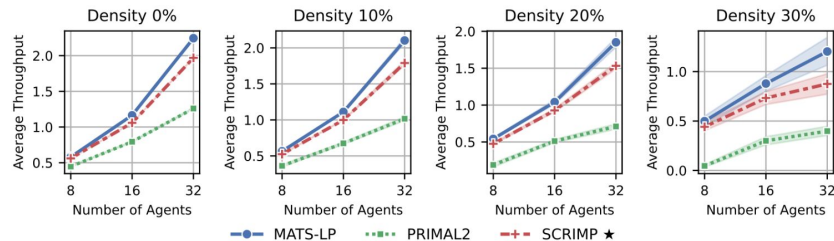
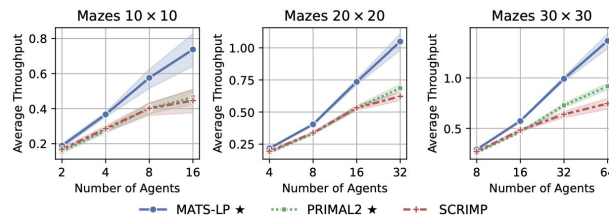
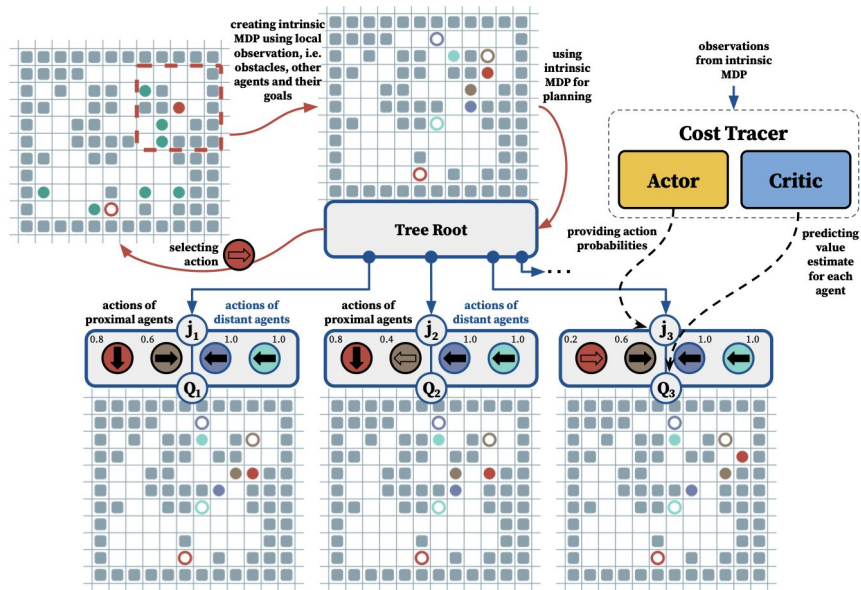
Online training statements	Supervised	GPT-f	Evariste miniF2F-valid
miniF2F-valid	38.5	47.3	58.6 [†]
miniF2F-test	35.3	36.6	41.0
miniF2F-curriculum	20.8	30.6	32.1
Train time (A100 days)	50	2000	1360

Faster sorting algorithms discovered using deep RL



70% for sequences of a length of five and roughly 1.7% for sequences exceeding 250,000 elements.

Decentralized Multi-Agent Pathfinding



Multi-Agent Starcraft

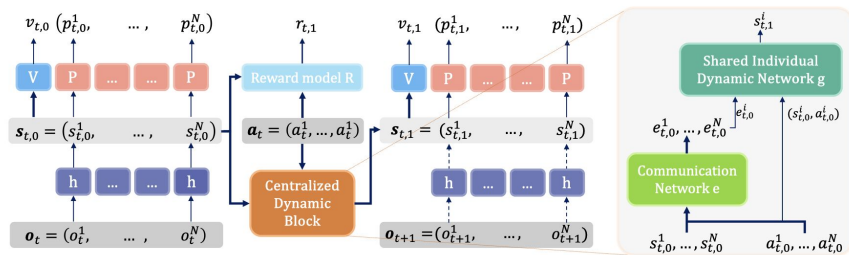
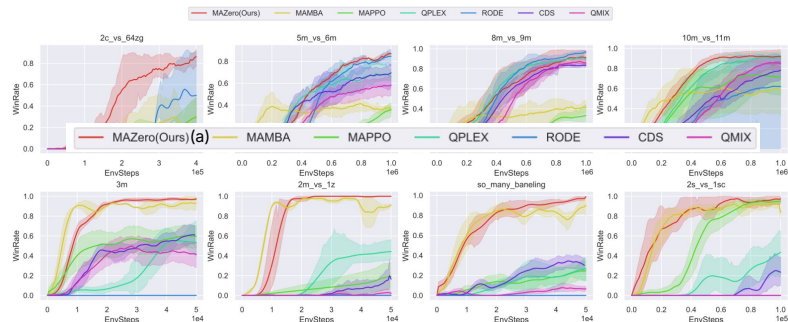


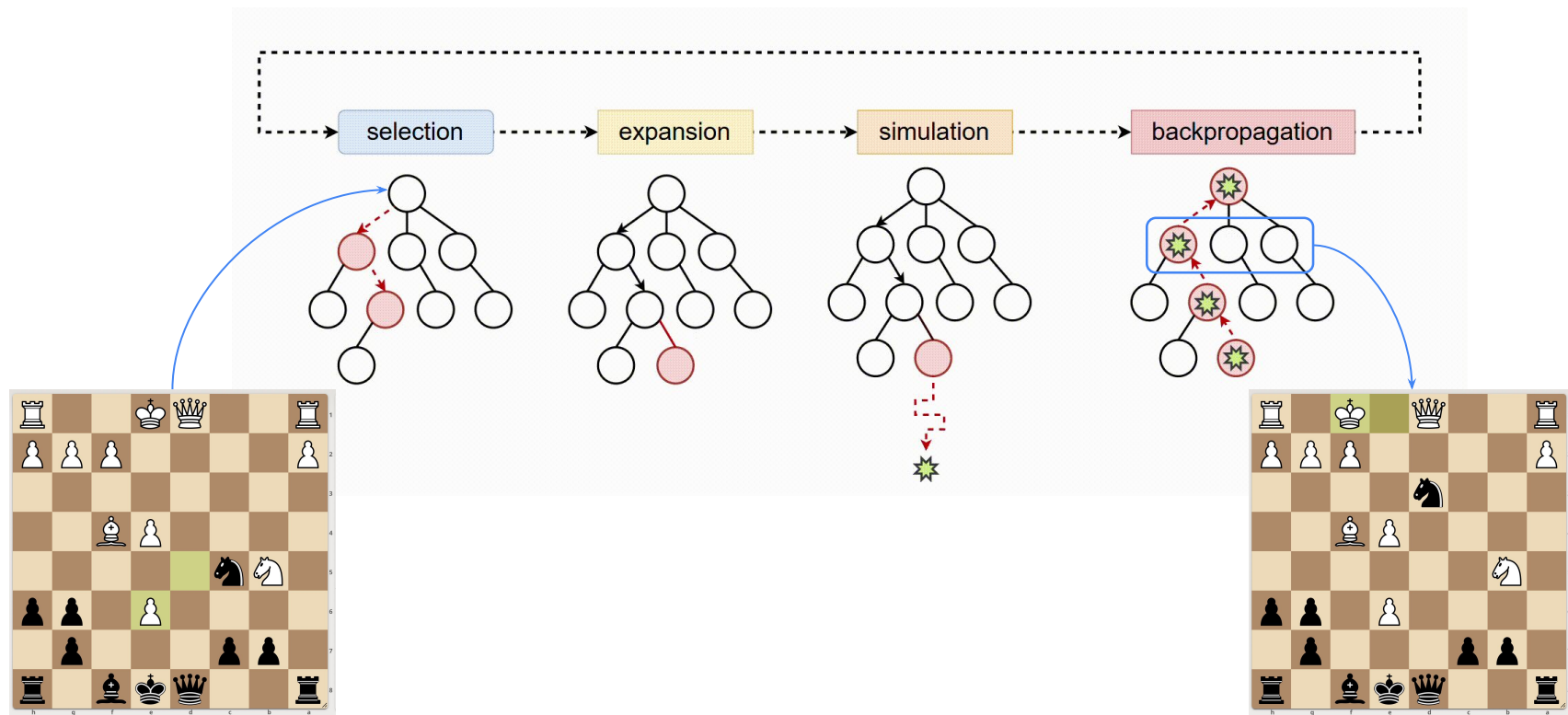
Figure 2: **MAZero model structure** Given the current observations $\mathbf{o}_t^i$ for each agent, the model separately maps them into local hidden states $\mathbf{s}_{t,0}^i$ using a shared representation network h . Value prediction $v_{t,0}$ is computed based on the global hidden state $\mathbf{s}_{t,0}$ while policy priors $p_{t,0}^i$ are individually calculated for each agent using their corresponding local hidden states. Agents use the communication network e to access team information $\mathbf{e}_{t,0}^i$ and generate next local hidden states $\mathbf{s}_{t,1}^i$ via the shared individual dynamic network g , subsequently deriving reward $r_{t,1}$, value $v_{t,1}$ and policies $p_{t,1}^i$. During the training stage, real future observations $\mathbf{o}_{t+1}$ can be obtained to generate the target for the next hidden state, denoted as $\mathbf{s}_{t+1,0}$.



02

Классический Монте-Карло
поиск по дереву

Background: Monte-Carlo Tree Search (MCTS)



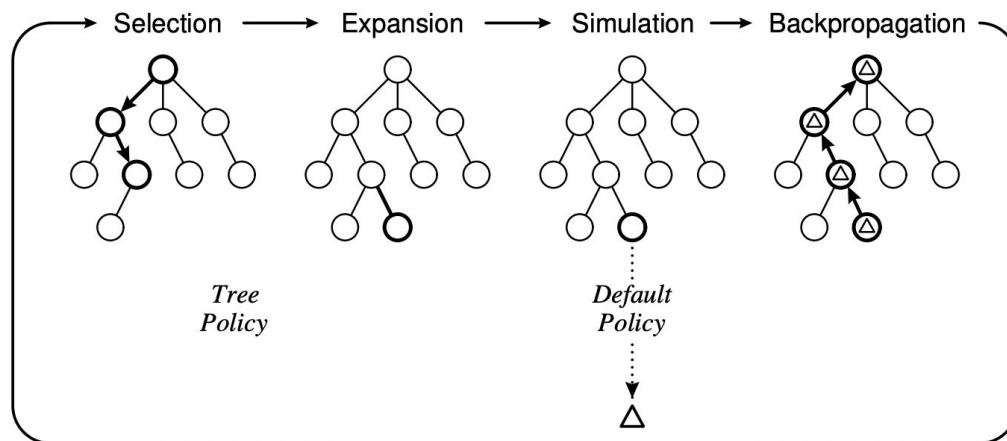
Background: Monte-Carlo Tree Search (MCTS)



How to make single action?

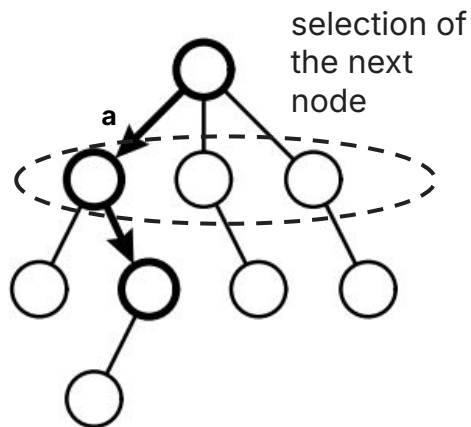
Algorithm 1 General MCTS approach.

```
function MCTSSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
    BACKUP( $v_l, \Delta$ )  
  return  $a(\text{BESTCHILD}(v_0))$ 
```



A Survey of Monte Carlo Tree Search Methods

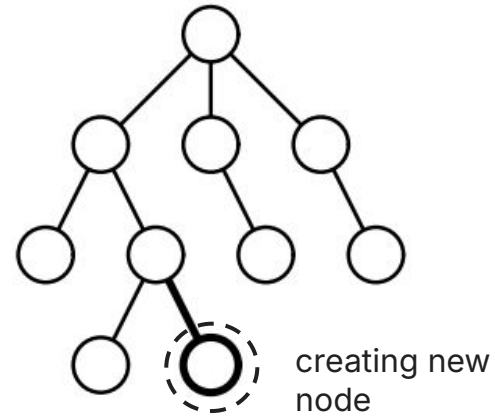
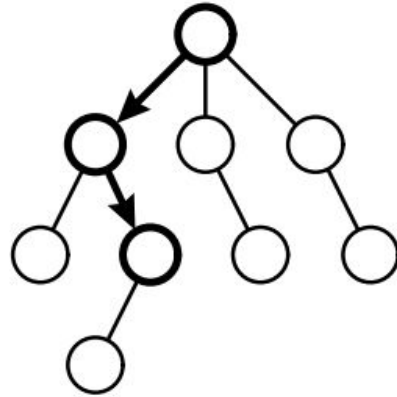
MCTS: Selection



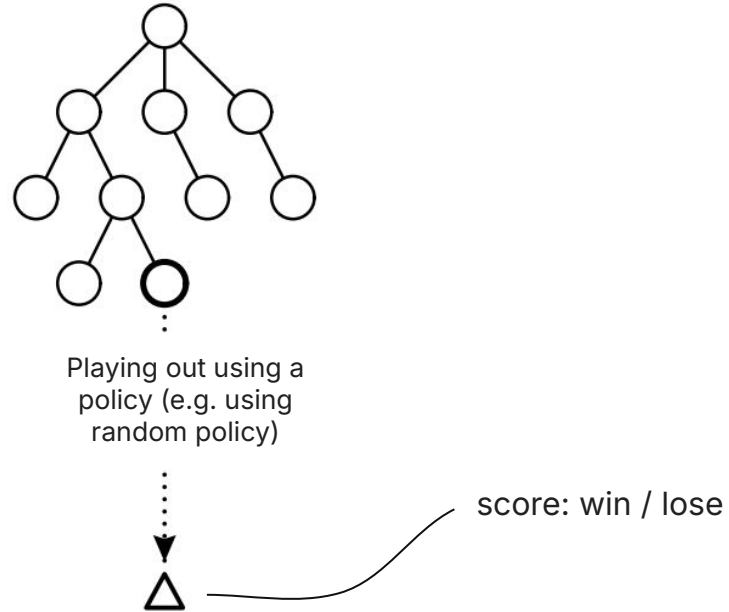
$$UCT(s, a) = Q(s, a) + c \cdot \sqrt{\frac{\overbrace{\ln(\sum_b N(s, b))}^{\text{number of visits of parent node}}}{1 + \underbrace{N(s, a)}_{\text{number of visits for node}}}}$$

$\frac{w_a}{N(s, a)}$ % wins

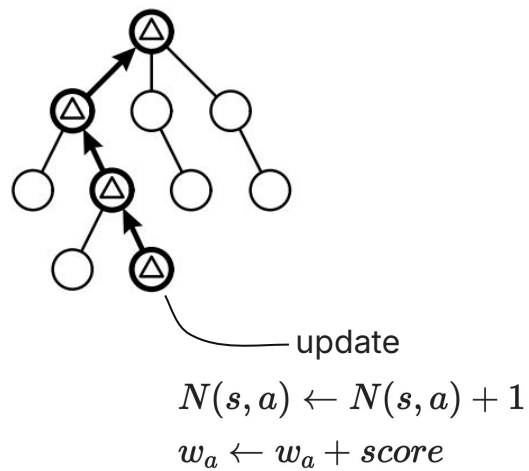
MCTS: Expansion



MCTS: Simulation



MCTS: Backpropagation

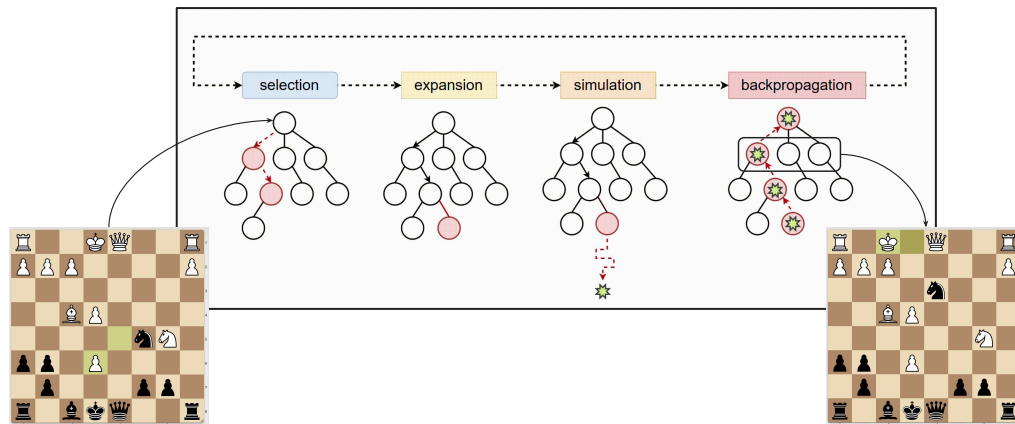


Monte-Carlo Tree Search

We call Algorithm 1 for each individual action in the episode

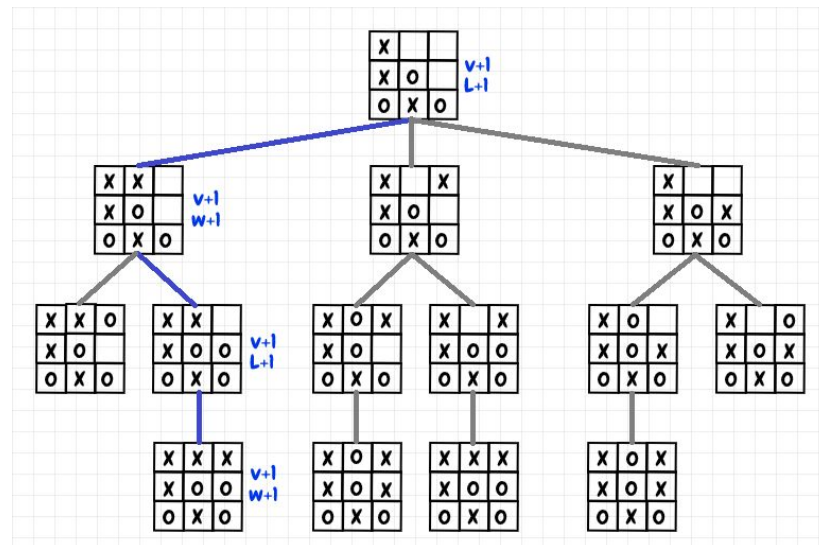
Algorithm 1 General MCTS approach.

```
function MCTSSEARCH( $s_0$ )  
  create root node  $v_0$  with state  $s_0$   
  while within computational budget do  
     $v_l \leftarrow \text{TREEPOLICY}(v_0)$   
     $\Delta \leftarrow \text{DEFAULTPOLICY}(s(v_l))$   
     $\text{BACKUP}(v_l, \Delta)$   
  return  $a(\text{BESTCHILD}(v_0))$ 
```



Classic MCTS Summary:

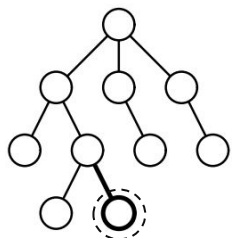
- Smart alternative to exhaustive search
- MCTS has been used for various tasks, such as planning, scheduling, and physical simulations (e.g., the HOOT algorithm)
- It gained the most recognition for its application to the game of Go
- It can show strong performance with a high-quality default policy (for limited action space)



03

Нейронный MCTS в AlphaZero

1) How to choose actions in the tree?



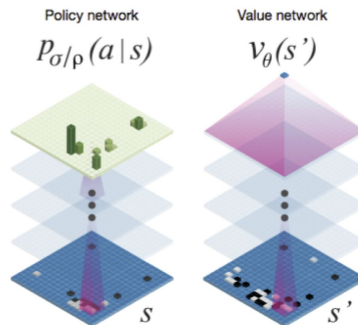
Q + U

The mean value of the next state

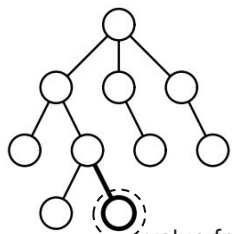
A function of **P** and **N** that increases if an action hasn't been explored much, relative to the other actions, or if the prior probability of the action is high

$$pUCT = Q(s, a) + P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right)$$

p is for prior probability



1) How to choose actions in the tree?



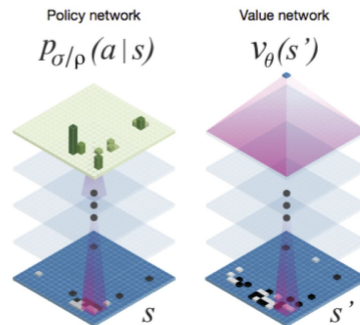
The mean value of the next state

$Q + U$

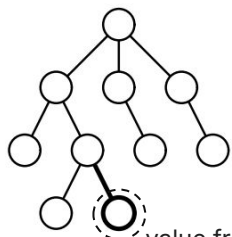
A function of P and N that increases if an action hasn't been explored much, relative to the other actions, or if the prior probability of the action is high

value from critic

$$pUCT = \underbrace{Q(s, a)}_{\text{value from critic}} + P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right)$$



1) How to choose actions in the tree?



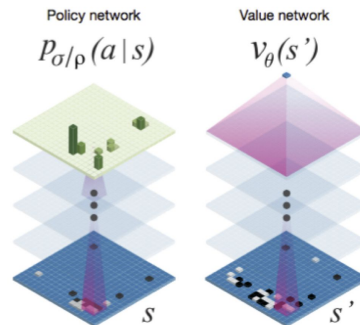
The mean value of the next state

$Q + U$

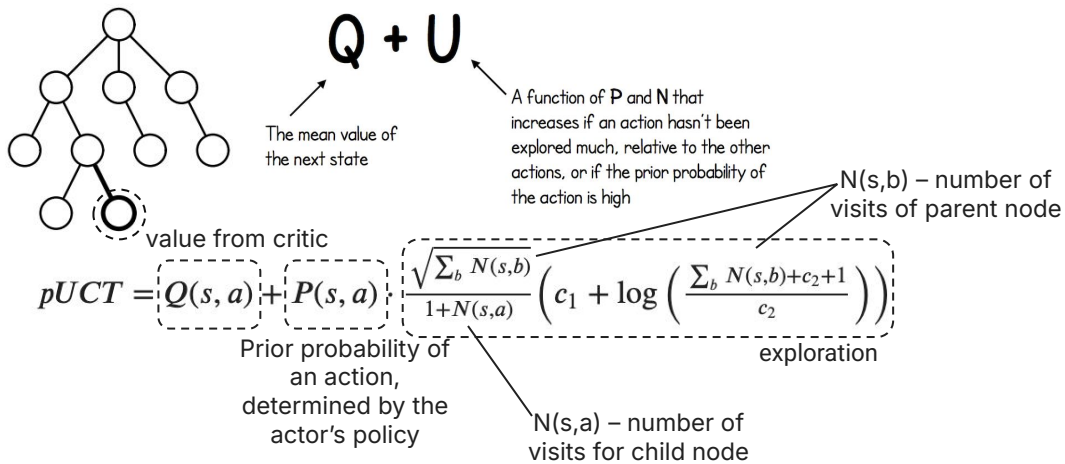
A function of P and N that increases if an action hasn't been explored much, relative to the other actions, or if the prior probability of the action is high

$$pUCT = \underbrace{Q(s, a)}_{\text{value from critic}} + \underbrace{P(s, a)}_{\text{Prior probability of an action, determined by the actor's policy}} \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right)$$

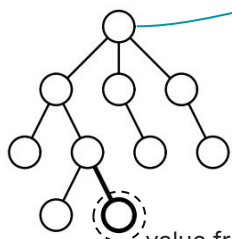
Prior probability of an action, determined by the actor's policy



1) How to choose actions in the tree?



1) How to choose actions in the tree?



The mean value of the next state

$Q + U$

A function of P and N that increases if an action hasn't been explored much, relative to the other actions, or if the prior probability of the action is high

+ Extra Dirichlet noise for Root node during training. Only for root node

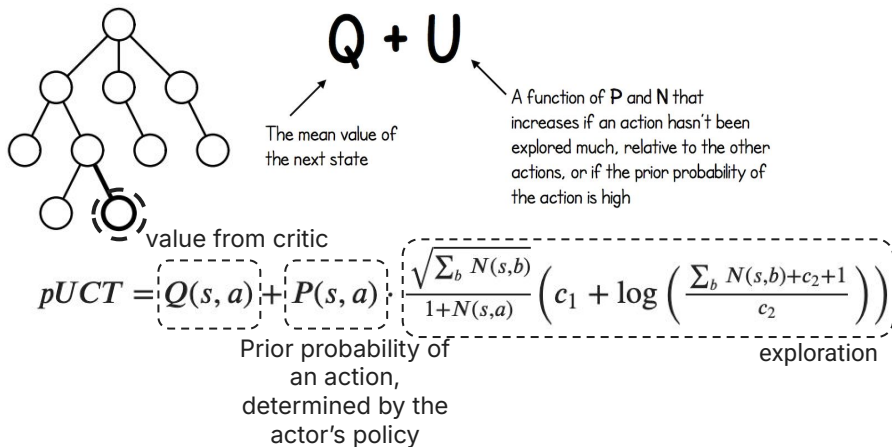
value from critic

$$pUCT = Q(s, a) + P(s, a) \left(\frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \left(c_1 + \log \left(\frac{\sum_b N(s, b) + c_2 + 1}{c_2} \right) \right) \right)$$

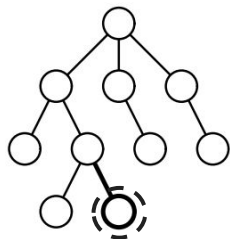
Prior probability of an action, determined by the actor's policy

exploration

1) How to choose actions in the tree?



2) We continue until we find an unexplored node



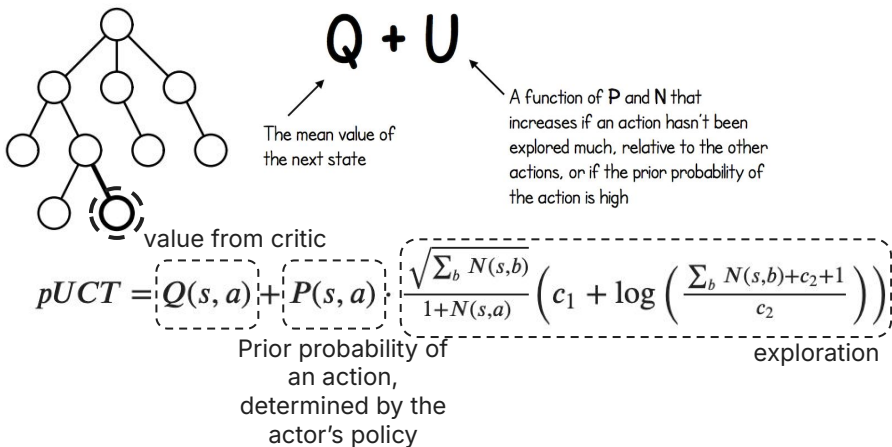
The game state of the leaf node is passed into the neural network, which outputs predictions about two things:

p Move probabilities

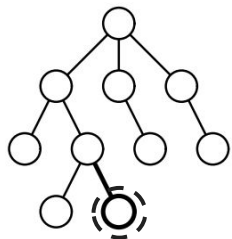
v Value of the state (for the current player)

The move probabilities p are attached to the new feasible actions from the leaf node

1) How to choose actions in the tree?



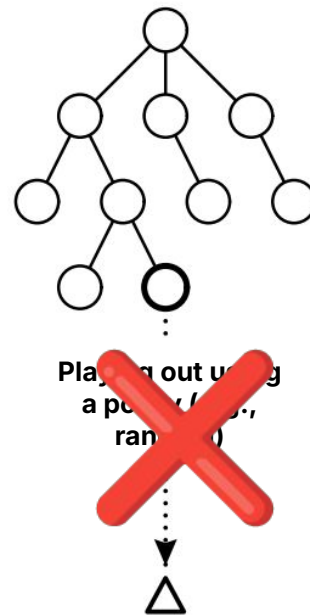
2) We continue until we find an unexplored node



p Move probabilities

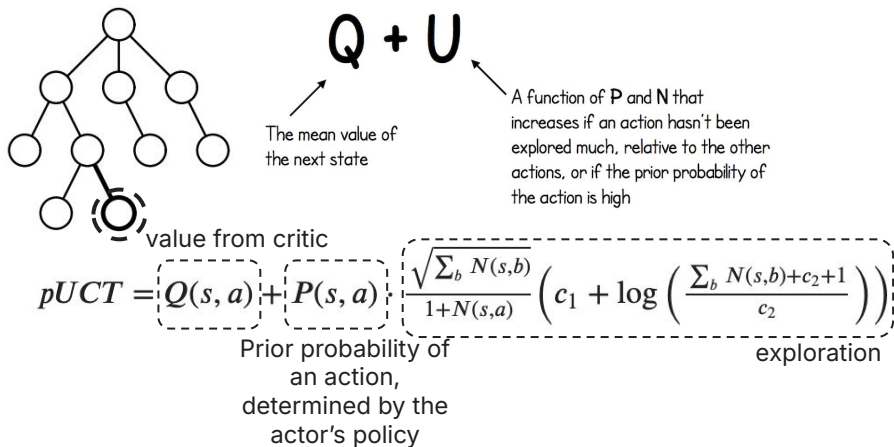
v Value of the state (for the current player)

The move probabilities p are attached to the new feasible actions from the leaf node

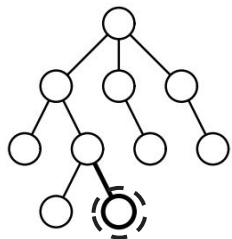


No simulation!
Just using the output of the critic for state of this node

1) How to choose actions in the tree?



2) We continue until we find an unexplored node



The game state of the leaf node is passed into the neural network, which outputs predictions about two things:

p Move probabilities

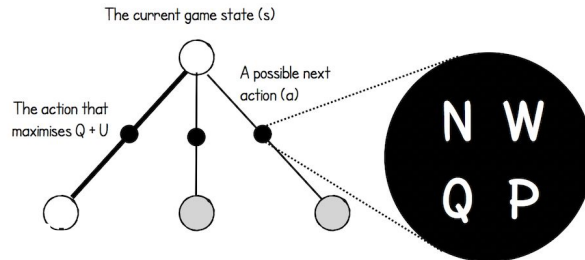
v Value of the state (for the current player)

The move probabilities p are attached to the new feasible actions from the leaf node

3) Backpropagation

Each edge that was traversed to get to the leaf node is updated as follows:

$$\begin{aligned} N &\rightarrow N + 1 \\ W &\rightarrow W + v \\ Q &= W / N \end{aligned}$$



How to choose the best action after constructing the tree?

...then select a move

After 1,600 simulations, the move can either be chosen:

Deterministically (for competitive play)

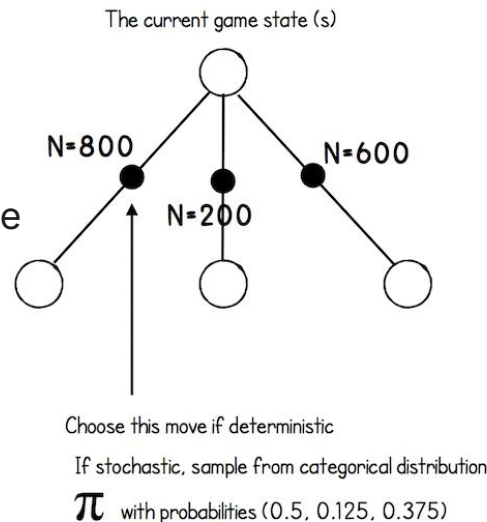
Choose the action from the current state a) the most explored subtree

Stochastically (for exploratory play)

Choose the action from the current state from the distribution

$$\pi \sim N^{1/\tau}$$

where τ is a temperature b) sampling from distribution



Tree search is a policy improvement operator!

After N simulations, we get a better policy.

How is it trained?

It's implicit curriculum learning!

SELF PLAY

Playing against
its own copy

Create a 'training set'

At each move, the following information is stored



State

π

The search probabilities
(from the MCTS)

Game results



The winner
(+1 if this player won, -1 if
this player lost - added once
the game has finished)

Improved by MCTS policy
(softmax of probabilities)

How is it trained?

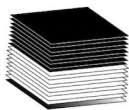
It's implicit curriculum learning!

SELF PLAY

Playing against its own copy

Create a 'training set'

At each move, the following information is stored



State

π

The search probabilities
(from the MCTS)

Game results



The winner
(+1 if this player won, -1 if
this player lost - added once
the game has finished)

RETRAIN NETWORK

Optimise the network weights learning

A TRAINING LOOP

Sample a mini-batch of 2048 positions from the last 500,000 games

Retrain the current neural network on these positions

- The game states are the input (see 'Deep Neural Network Architecture')

Loss function

Compares predictions from the neural network with the search probabilities and actual winner

PREDICTIONS $\begin{pmatrix} \mathbf{p} \\ \mathbf{v} \end{pmatrix}$

Cross-entropy + Mean-squared error + Regularisation

ACTUAL $\begin{pmatrix} \pi \\ \text{trophy} \end{pmatrix}$

After every 1,000 training loops, evaluate the network

$$l = (z - v)^2 - \pi^T \log \mathbf{p} + c ||\theta||^2$$

critic MSE actor cross-entropy L2 reg

Improved by MCTS policy
(softmax of probabilities)

How is it trained?

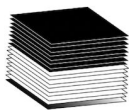
It's implicit curriculum learning!

SELF PLAY

Playing against its own copy

Create a 'training set'

At each move, the following information is stored



State

π

The search probabilities
(from the MCTS)

Game results



The winner
(+1 if this player won, -1 if
this player lost - added once
the game has finished)

RETRAIN NETWORK

Optimise the network weights learning

A TRAINING LOOP

Sample a mini-batch of 2048 positions from the last 500,000 games

Retrain the current neural network on these positions

- The game states are the input (see 'Deep Neural Network Architecture')

Loss function

Compares predictions from the neural network with the search probabilities and actual winner

$$\begin{array}{ccccc} \text{PREDICTIONS} & \begin{array}{c} \mathbf{p} \\ \mathbf{v} \end{array} & \begin{array}{c} \text{Cross-entropy} \\ + \\ \text{Mean-squared error} \\ + \\ \text{Regularisation} \end{array} & \begin{array}{c} \pi \\ \text{trophy} \end{array} & \text{ACTUAL} \end{array}$$

After every 1,000 training loops, evaluate the network

EVALUATE NETWORK

Test to see if the new network is stronger

Play 400 games between the latest neural network and the current best neural network

Both players use MCTS to select their moves, with their respective neural networks to evaluate leaf nodes

Latest player must win 55% of games to be declared the new best player



Policy estimation
(pitting against previous copy)

$$l = (z - v)^2 - \pi^T \log \mathbf{p} + c \|\theta\|^2$$

critic MSE

actor

cross-entropy

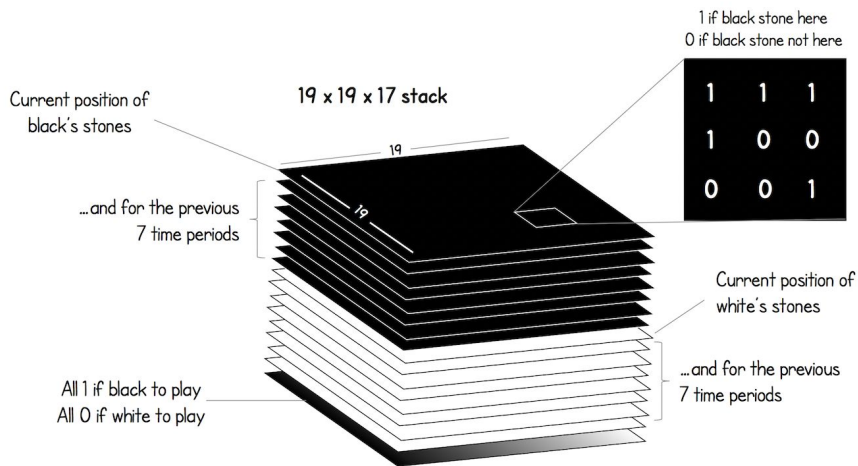
L2 reg

Improved by MCTS policy
(softmax of probabilities)

ResNet Encoder

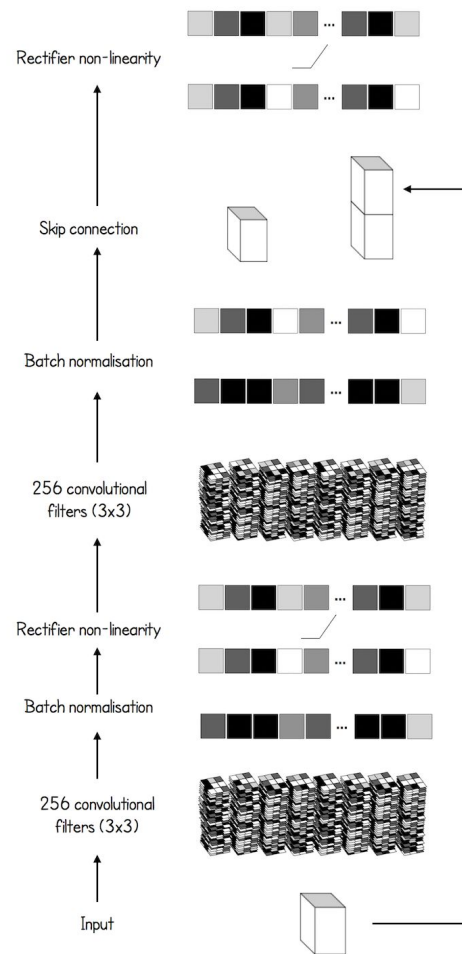
Game state encoding

WHAT IS A 'GAME STATE'



This stack is the input to the deep neural network

Default ResNet without
max-pooling



Results

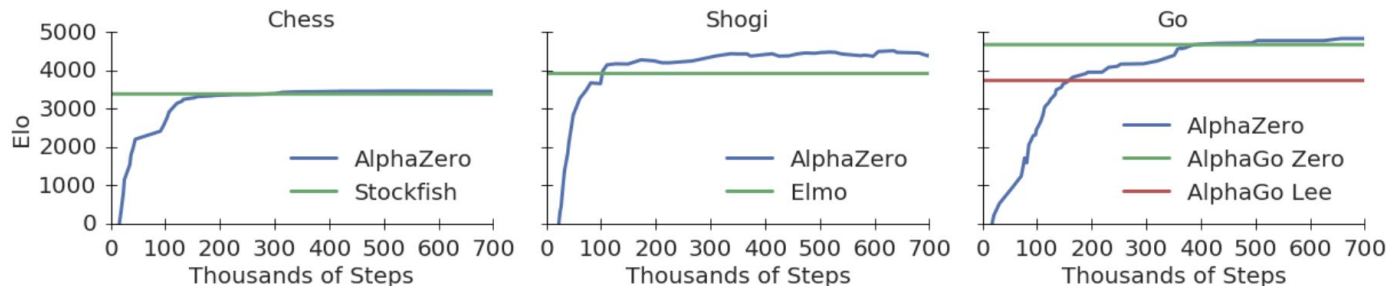
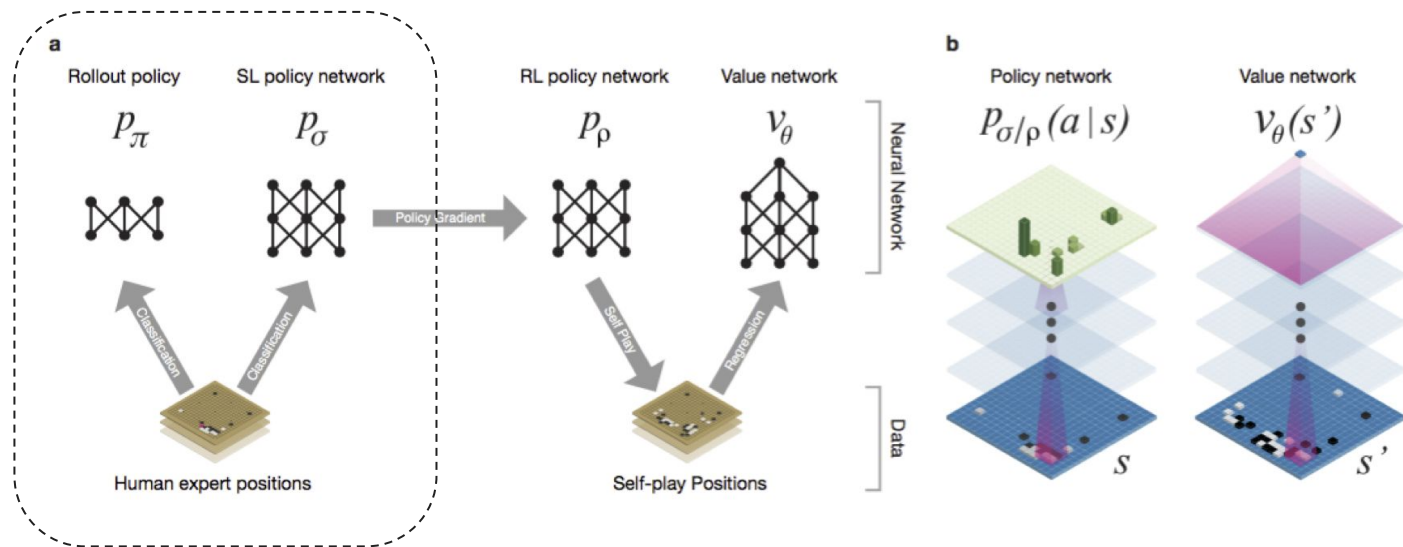


Figure 1: Training *AlphaZero* for 700,000 steps. Elo ratings were computed from evaluation games between different players when given one second per move. **a** Performance of *AlphaZero* in chess, compared to 2016 TCEC world-champion program *Stockfish*. **b** Performance of *AlphaZero* in shogi, compared to 2017 CSA world-champion program *Elmo*. **c** Performance of *AlphaZero* in Go, compared to *AlphaGo Lee* and *AlphaGo Zero* (20 block / 3 day) (29).

Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm

David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, Demis Hassabis

Difference with AlphaGo



Pretraining: Behavioral cloning on human experts data

+ many other tricks, such as the fast rollout policy, which were later dropped

AlphaZero Summary:

- Tabula rasa self-play training
- No human knowledge input
- ResNet encoder architecture
- Superhuman performance, mastery in chess, shogi, and Go
- Limitations?



AlphaZero Summary:

- Tabula rasa self-play training
- No human knowledge input
- ResNet encoder architecture
- Superhuman performance, mastery in chess, shogi, and Go
- Requires a simulator
- Only deterministic environments
- Simplified RL (no discount factor, etc.)
- Computationally expensive

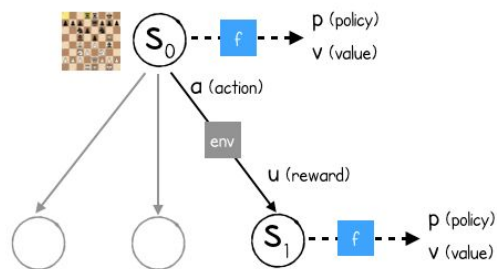


04

Нейронный MCTS в MuZero

AlphaZero vs MuZero

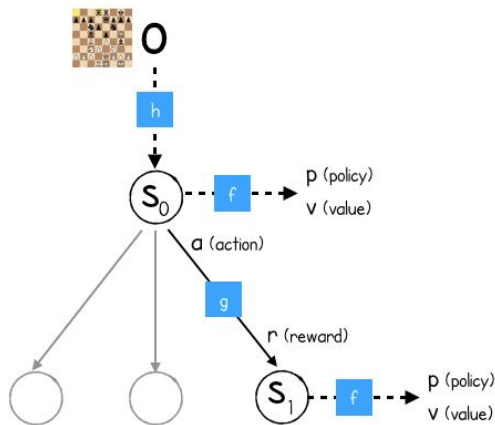
AlphaZero



AlphaZero has 1 network

prediction f : $s \rightarrow p, v$

MuZero



MuZero has 3 networks

	from	to
prediction	f :	$s \rightarrow p, v$
dynamics	g :	$s, a \rightarrow r, s$
representation	h :	$o \rightarrow s$

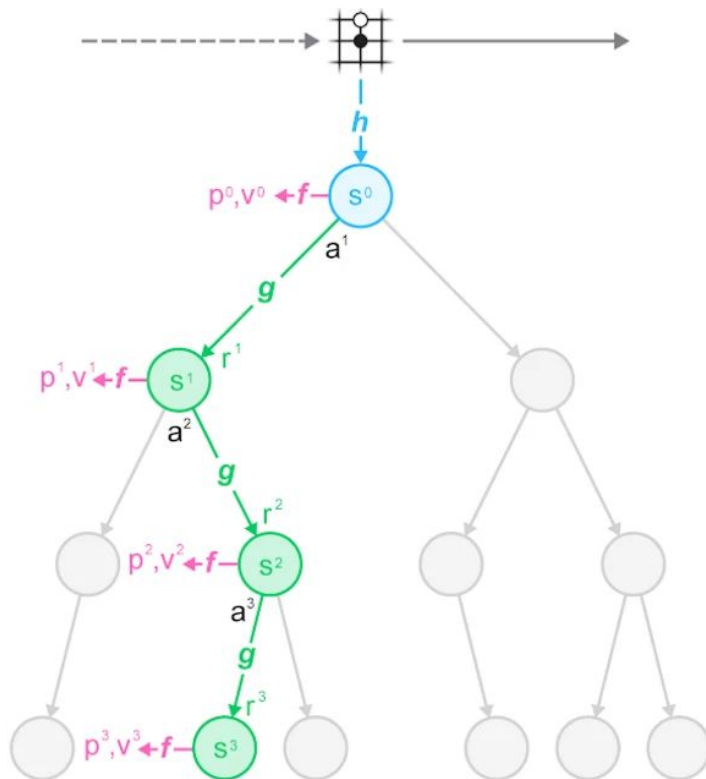
→ Requires a simulator

MuZero

representation $h_{\theta}(o_1, \dots, o_t) = s^0$

prediction $f_{\theta}(s^k) = \mathbf{p}^k, v^k$

dynamics $g_{\theta}(s^{k-1}, a^k) = r^k, s^k$



pUCT for Reinforcement Learning Environments

$$U(s, a) = r(s, a) + \gamma \cdot v(s') + \overset{\text{exploration}}{\boxed{c \cdot p(s, a)}}$$

c is still that large formulae...

$$U(s, a) = \overset{\text{extra normalization}}{\boxed{\frac{r(s, a) + \gamma \cdot v(s') - q_{min}}{q_{max} - q_{min}}}} + c \cdot p(s, a)$$

MuZero Training

Model

$$\left. \begin{aligned} s^0 &= h_\theta(o_1, \dots, o_t) \\ r^k, s^k &= g_\theta(s^{k-1}, a^k) \\ p^k, v^k &= f_\theta(s^k) \end{aligned} \right\} p^k, v^k, r^k = \mu_\theta(o_1, \dots, o_t, a^1, \dots, a^k)$$

h produces s^0 , g is used to generate r^k and s^k ,
 s^0 and s^k are implicit targets

Search

$$\nu_t, \pi_t = MCTS(s_t^0, \mu_\theta)$$

$$a_t \sim \pi_t$$

Learning Rule

$$p_t^k, v_t^k, r_t^k = \mu_\theta(o_1, \dots, o_t, a_{t+1}, \dots, a_{t+k})$$

$$z_t = \begin{cases} u_T & \text{for games} \\ u_{t+1} + \gamma u_{t+2} + \dots + \gamma^{n-1} u_{t+n} + \gamma^n \nu_{t+n} & \text{for general MDPs} \end{cases}$$

n-step bootstrapping

$$l_t(\theta) = \sum_{k=0}^K l^r(u_{t+k}, r_t^k) + l^v(z_{t+k}, v_t^k) + l^p(\pi_{t+k}, p_t^k) + c \|\theta\|^2$$

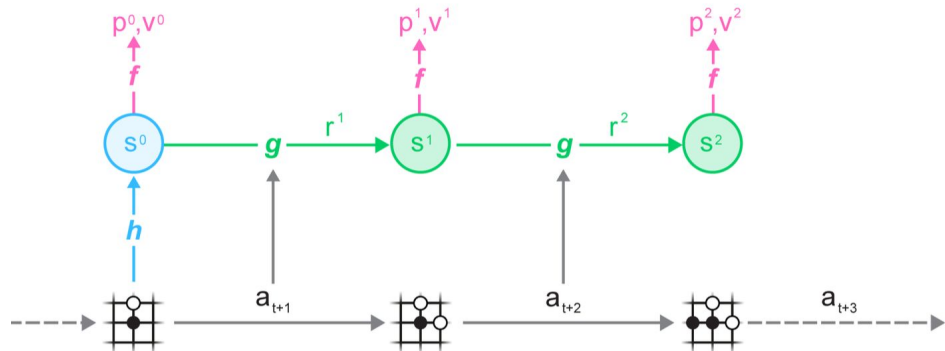
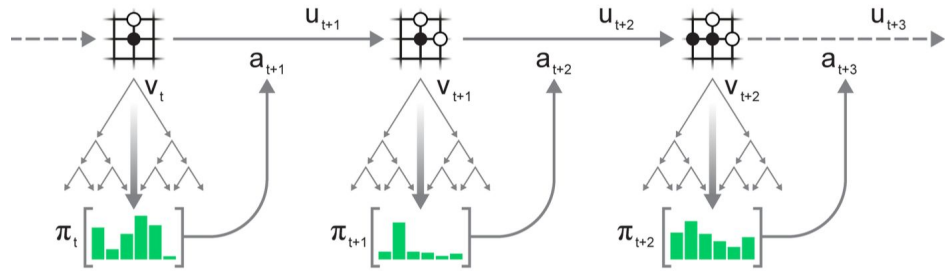
reward critic actor L2 reg

Losses

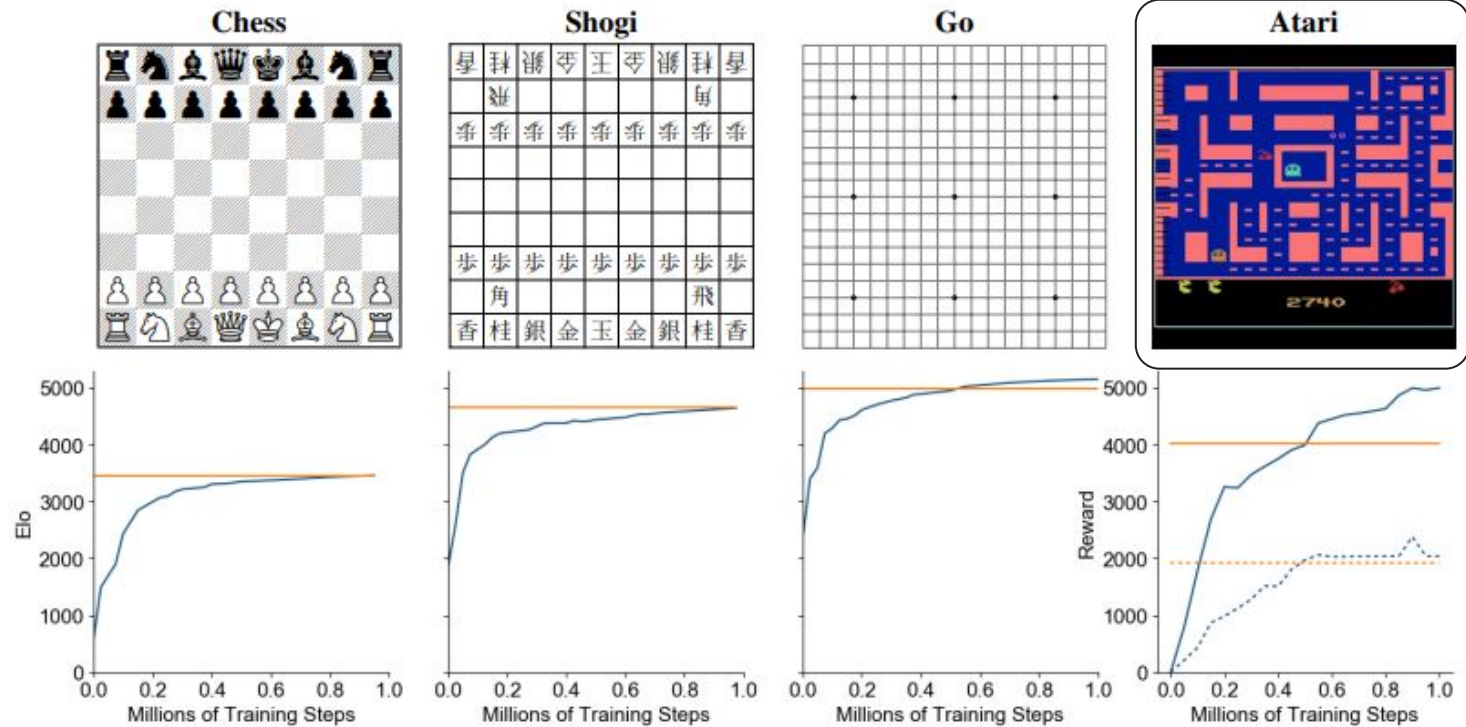
$$l^r(u, r) = \begin{cases} 0 & \text{for games} \\ \phi(u)^T \log \mathbf{r} & \text{for general MDPs} \end{cases}$$

$$l^v(z, q) = \begin{cases} (z - q)^2 & \text{for games} \\ \phi(z)^T \log \mathbf{q} & \text{for general MDPs} \end{cases} \quad (\phi \text{ is a cross-entropy over bins})$$

$$l^p(\pi, p) = \pi^T \log \mathbf{p}$$



Results



Comparison

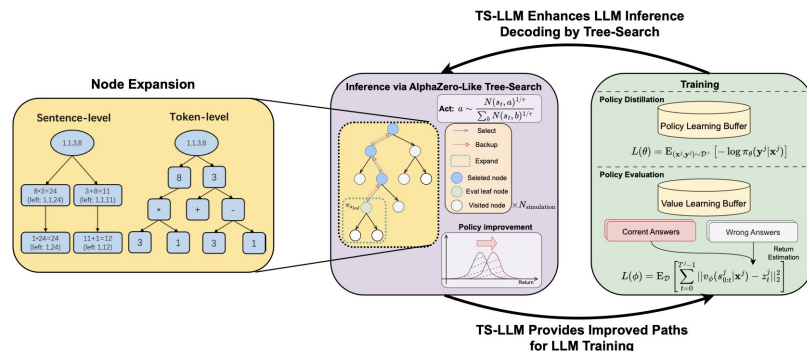
Agent	Median	Mean	Env. Frames
Ape-X	434.1%	1695.6%	22.8B
R2D2	1920.6%	4024.9%	37.5B
MuZero	2041.1%	4999.2%	20.0B
IMPALA	191.8%	957.6%	200M
Rainbow	231.1%	–	200M
UNREAL	250%	880%	200M
LASER	431%	–	200M
MuZero Reanalyse	731.1%	2168.9%	200M

05

Применение MCTS для языковых моделей

AlphaZero-like tree-search for LLMs

- Uses an AlphaZero-style framework where an LLM acts as both the policy and the value network inside MCTS
- At each step, the LLM proposes possible actions (text continuations, reasoning steps, or plans) and estimates their expected outcomes
- The MCTS uses these priors and values to efficiently explore the reasoning/action space, balancing exploitation of high-probability paths with exploration of uncertain options
- This closed-loop setup allows the LLM to plan more coherent long-range outputs, improving performance on tasks like code generation, multi-step reasoning



Task	Category	Train/test size	Search Space Hyperparameters		
			Node	Tree Max width	Tree Max depth
GSM8k	Mathematical Reasoning	7.5k / 1.3k	Sentence	6	8
Game24	Mathematical Planning	1.0k / 0.3k	Sentence	20	4
PrOntoQA	Logical Reasoning	4.5k / 0.5k	Sentence	6	15
RLHF	Alignment	30k / 3k	Token	50	64
Chess Endgame	Decision Making	0.1M/0.6k	Sentence	5	50

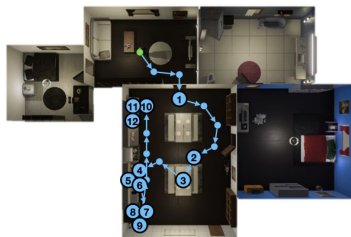
AlphaZero-like tree-search for LLMs

- LLaMA-SFT and LLaMA-V refer to the trained policy and value, LLaMA and GPT-3.5 (ToT) refer to the prompt-based model for policy or value
- LLaMA-V dominates compared with prompt-based value

Task	Policy	Value	Accuracy(%)
GSM8K	GPT-3.5	GPT-3.5 (ToT)	72.7
	GPT-3.5	LLaMA-V (Ours)	74.0
	LLaMA-SFT	LLaMA (ToT)	37.4
	LLaMA-SFT	GPT-3.5 (ToT)	45.8
	LLaMA-SFT	LLaMA-V (Ours)	52.5
Game24	GPT-3.5	GPT-3.5 (ToT)	15.5
	GPT-3.5	LLaMA-V (Ours)	19.1
	LLaMA-SFT	LLaMA (ToT)	9.2
	LLaMA-SFT	GPT-3.5 (ToT)	21.0
	LLaMA-SFT	LLaMA-V (Ours)	64.8

LLM as commonsense knowledge for large-scale task planning

- LLM-MCTS, which treats an LLM as both a world model and a policy within an MCTS planner
- The LLM-induced world model provides commonsense priors and the LLM policy guides exploration, dramatically improving large-scale task planning over using an LLM alone or vanilla MCTS
- Experiments on complex planning tasks (e.g. arithmetic planning, travel planning) show LLM-MCTS greatly outperforms baselines



Instruction: Put one plate inside the kitchen cabinet and one chicken inside microwave.
 1: Walk to kitchen
 2: Walk to plate
 3: Grab plate
 4: Walk to kitchen cabinet
 5: Open kitchen cabinet
 6: Put plate inside kitchen cabinet
 7: Move fridge
 8: Open fridge
 9: Grab chicken
 10: Walk to microwave
 11: Open microwave
 12: Put chicken inside microwave

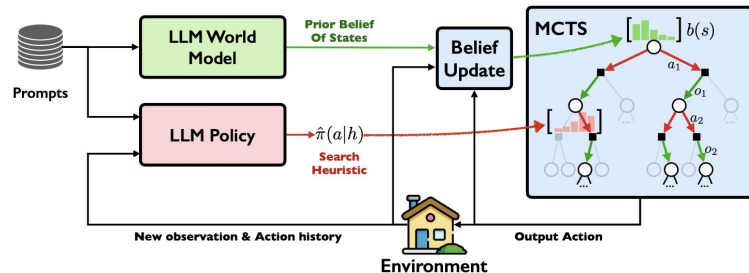
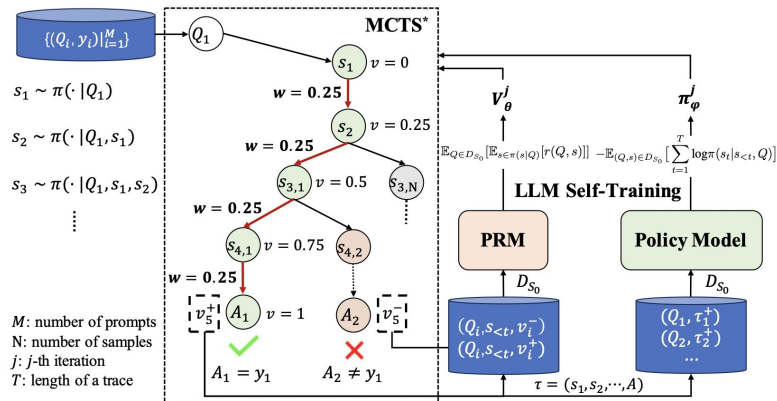


Table 1: Main results: mean $\pm$ standard error of success rate (%)

Method	Seen Home				
	Simple	Comp.	NovelSimple	NovelComp.(2)	NovelComp.(3)
UCT [21]	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
finetuned GPT2 policy [22]	81.3 $\pm$ 2.4	59.0 $\pm$ 6.7	41.2 $\pm$ 7.1	30.9 $\pm$ 2.8	2.3 $\pm$ 1.5
GPT3.5 Policy [18]	83.4 $\pm$ 6.8	47.0 $\pm$ 7.8	74.3 $\pm$ 4.0	48.2 $\pm$ 8.8	5.4 $\pm$ 2.0
GPT3.5-MCTS (Ours)	91.4 $\pm$ 3.3	71.2 $\pm$ 6.2	88.1 $\pm$ 4.3	72.6 $\pm$ 6.9	33.6 $\pm$ 3.1
Method	Unseen Home				
	Simple	Comp.	NovelSimple	NovelComp.(2)	NovelComp.(3)
UCT [21]	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0
finetuned GPT2 policy [22]	65.5 $\pm$ 3.4	39.9 $\pm$ 5.2	33.4 $\pm$ 6.4	12.8 $\pm$ 3.9	1.1 $\pm$ 0.9
GPT3.5 Policy [18]	74.3 $\pm$ 5.0	43.3 $\pm$ 4.0	67.8 $\pm$ 4.9	54.0 $\pm$ 3.0	6.9 $\pm$ 2.1
GPT3.5-MCTS (Ours)	82.9 $\pm$ 3.2	71.9 $\pm$ 5.6	79.3 $\pm$ 3.3	70.4 $\pm$ 6.4	38.8 $\pm$ 3.4

ReST-MCTS*

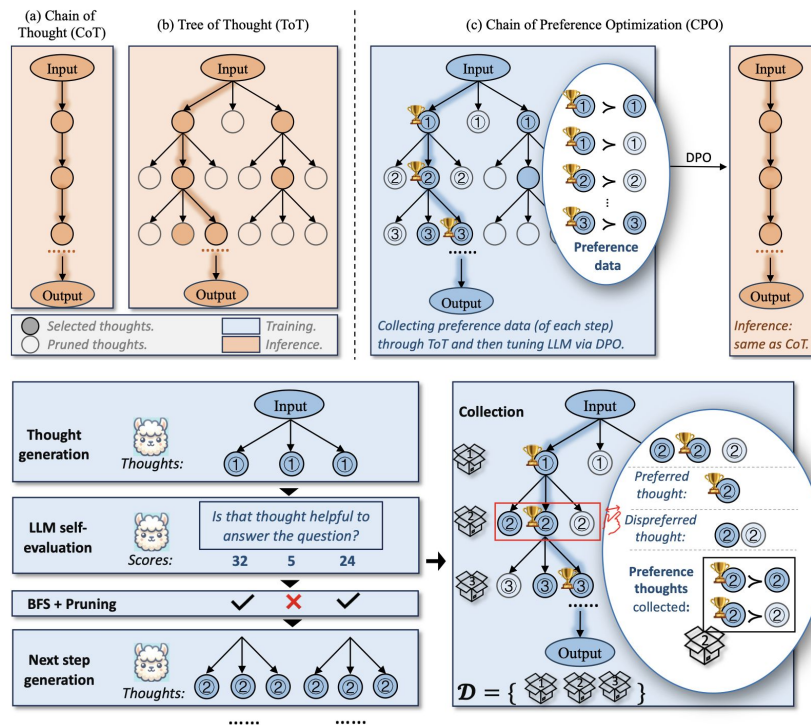
- Proposes a self-training framework that uses a modified MCTS (called MCTS*) guided by learned per-step “process” reward models
- Given correct final answers, ReST-MCTS* runs MCTS* to infer which intermediate reasoning steps lead to the solution, automatically labeling each step’s reward
- These high-quality reasoning traces and value targets are then used to fine-tune the LLM
- Compared to prior self-training methods (best-of-N, basic tree-of-thought, etc.), this reward-guided tree search yields more accurate reasoning and better LLM performance



Method	Reasoning Policy	Reward Guidance	
	Method	Value Label	Train
STaR [4]	CoT+Reflexion	Final outcome reward annotated by ground-truth answer	✗
ReST ^{EM} [6] / RFT [5] / RPO [11]	CoT		✗
Verify Step-by-Step [1]	Best-of-N	Per-step process reward annotated by human	✓
MATH-SHEPHERD [12] / pDPO [13]	Best-of-N	Per-step process reward inferred from random rollout	✓
TS-LLM [14]	MCTS	Per-step process reward inferred from TD- λ [15]	✓
V-STaR [7]	CoT	Final outcome reward generated by multi-iteration LLMs	✓
Self-Rewarding [16]	CoT	Final outcome reward generated and judged by LLMs	✗
ReST-MCTS* (Ours)	MCTS*	Per-step process reward inferred from tree search (MCTS*)	Multi-Iter

Chain of Preference Optimization

- Uses a tree-of-thought (ToT) search to discover high-quality reasoning paths, then fine-tunes the LLM (CoT) to follow those paths
- Specifically, ToT (an MCTS-based search over reasoning chains) is used to generate preference data for each reasoning step
- The LLM is then trained so that its own chain-of-thought aligns with the preferred steps found by the tree search
- This chain-of-preference optimization (CPO) lets the LLM achieve ToT-level performance using only standard CoT inference, greatly improving reasoning on QA, fact verification, arithmetic, etc., without the huge inference cost of ToT



GDP-ZERO

- Introduces GDP-ZERO (Goal-Oriented Dialogue Policy), which uses open-loop MCTS at inference time for dialogue planning
- An LLM is prompted in the loop to simulate future user/system utterances, act as a policy prior and value function, and assess task progress. By running MCTS with these LLM rollouts (rather than training any model), GDP-ZERO selects dialogue actions that lead to better outcomes.
- On the PersuasionForGood task, GDP-ZERO's LLM-driven MCTS planning yielded responses preferred over vanilla ChatGPT, demonstrating the power of MCTS-guided reasoning

... some dialogue history

Q: Do you ever donate to charities?

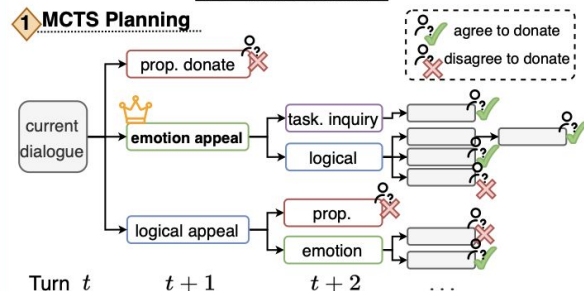
9: If I'm in the right place at the right time or am given an opportunity.

ChatGPT

🔊 🌀: Save the Children is a wonderful charity to donate to.
Would you be interested in donating to the charity?

V.S.

GDP-Zero+ChatGPT



2 Prediction

🔗 [emotion appeal]: It's wonderful to see people like you who care so much about helping others! *It's important to support causes that are close to our hearts. Children are ...*

GDP-ZERO

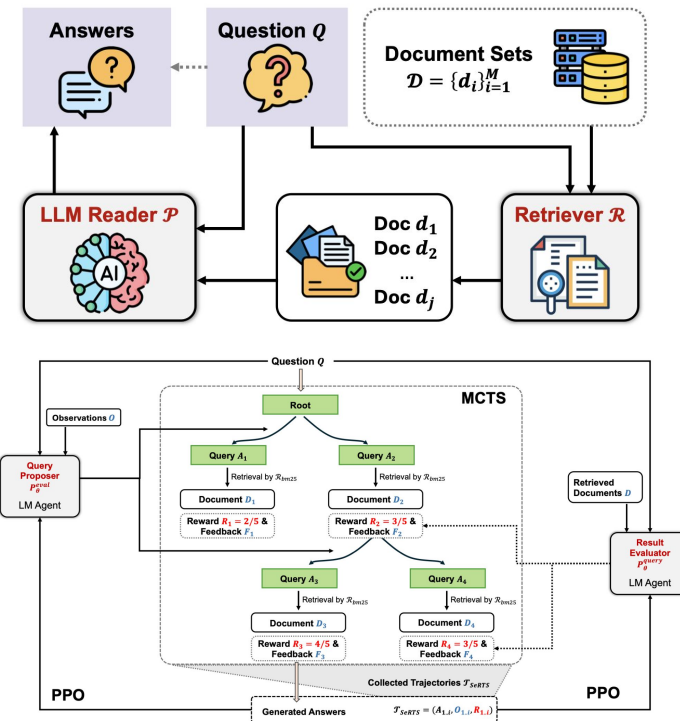
- Interactive evaluation using RAP, ChatGPT, and GDP-ZERO as planners, with ChatGPT used for response generation/backbone
- RAP is the current state-of-the-art system on P4G, using trained modules with fixed dialogue policies derived from expert knowledge.
- GDP-ZERO achieves the best performance across all metrics related to persuasiveness

The chatbot...	RAP	ChatGPT	GDP-Zero
donation prob. $\uparrow$	0.72 ± 0.38	0.73 ± 0.38	0.79 ± 0.37
inc. donation intent $\uparrow$	4.08 ± 0.68	3.77 ± 0.90	$4.30 \pm 0.71^{**}$
strong argument $\uparrow$	3.89 ± 0.97	3.91 ± 0.99	$4.28 \pm 0.74^*$
is convincing $\uparrow$	4.11 ± 0.74	4.10 ± 0.70	$4.38 \pm 0.66^{**}$
diverse strategy $\uparrow$	3.98 ± 0.80	3.83 ± 1.03	3.95 ± 0.82
is manipulative $\downarrow$	2.64 ± 1.36	2.96 ± 1.38	$2.29 \pm 1.33^{**}$
is natural $\uparrow$	4.25 ± 0.68	4.03 ± 0.65	$4.38 \pm 0.62^{**}$
is relevant $\uparrow$	$4.64 \pm 0.54^*$	4.31 ± 0.86	4.59 ± 0.49
is coherent $\uparrow$	4.28 ± 0.65	4.06 ± 0.89	$4.42 \pm 0.49^*$

Crowdworker	They help to provide basic health care, provide disaster relief, and help to ensure their rights to education and safety. How much would you like to donate ? Your donation will be deducted from your task payment.
ChatGPT	Save the Children provides a variety of services to children in need, including education, healthcare, and protection from harm. They also work with communities to help them become more self-sufficient and better able to care for their children.
GDP-ZERO	Save the Children provides support in a variety of ways. They help create safe spaces for children, provide education programs and materials, and offer medical care and support. They also advocate for policy changes that benefit children's rights around the world. Your donation would go directly to help these programs rather than promotion.

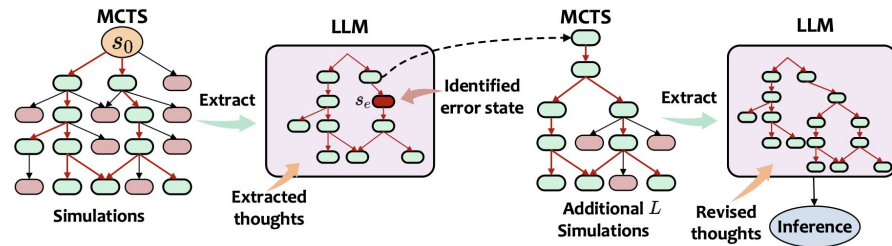
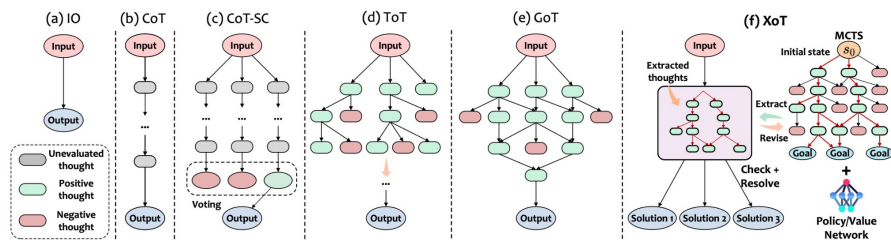
SeRTS

- Proposes a retrieval method combining LLMs and MCTS
- SeRTS uses MCTS to plan document retrieval: at each step, the LLM suggests retrieval actions, and the tree search evaluates their reward (relevance/factuality)
- Crucially, SeRTS then fine-tunes the LLM with PPO using the trajectories found by the tree search as feedback
- In biomedical QA, this self-rewarding MCTS loop significantly improves retrieval quality over BM25 and other baselines



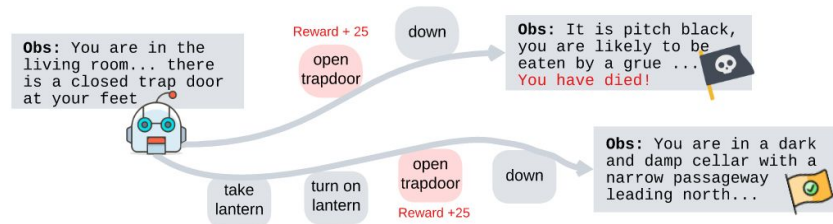
Everything of Thoughts (XoT)

- XoT augments an LLM's reasoning by integrating an MCTS-based planner. Specifically, it uses a pretrained value model and MCTS to revise the LLM's chain-of-thought outputs
- The MCTS-LLM "collaborative thought revision" allows the model to incorporate external domain knowledge and explore multiple reasoning paths
- On tasks like the 24 game, 8-puzzle, and Pocket Cube, XoT's MCTS-informed reasoning greatly improves over standard CoT, yielding higher accuracy and more efficient problem-solving



MCTS with LLMs for text-based game agents

- Uses an LLM as a world model and policy within an MCTS planner to act in interactive text-based games
- The LLM simulates possible action sequences and predicts game outcomes; MCTS explores these paths to find high-reward strategies
- This planning loop enables the agent to reason ahead, handle long-horizon dependencies, and outperform greedy or single-step LLM baselines

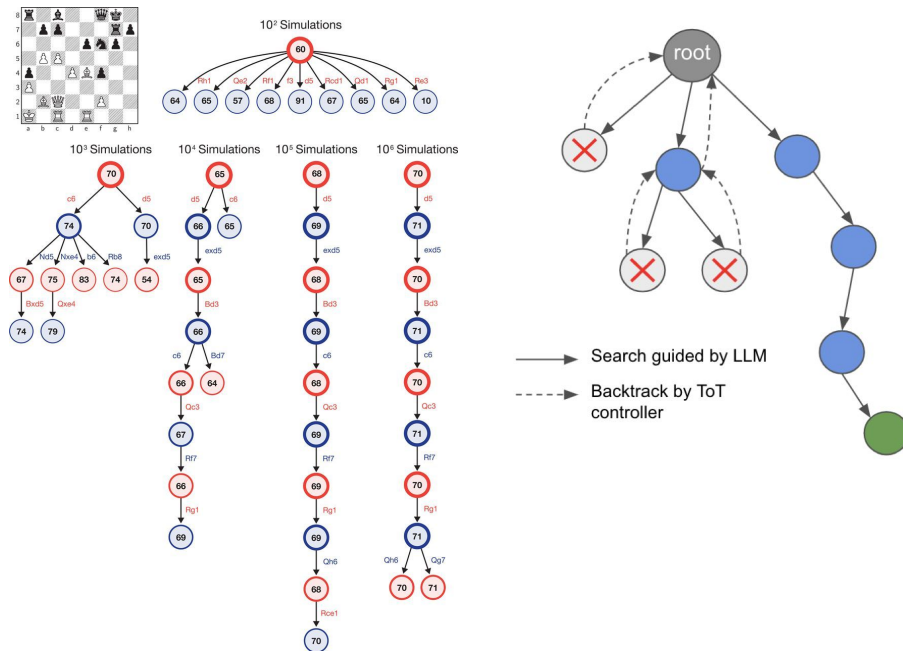


$$a^* = \arg \max_{a \in \mathcal{A}(s)} \left[Q(s, a) + C_{uct} \cdot \sqrt{\frac{\ln N(s)}{N(s, a)}} \right]$$

	PUCT-RL		MC-LAVE-RL		MC-DML
	Tree Search	RL	Tree Search	RL	Tree Search
Iteration 1	31.9 ± 1.4	36.6 ± 1.0	30.4 ± 2.0	36.6 ± 1.0	48.66 ± 1.89
Iteration 2	35.8 ± 0.0	37.4 ± 1.0	36.1 ± 0.1	38.2 ± 0.8	N/A
Iteration 3	35.3 ± 0.2	39.0 ± 0.0	41.2 ± 0.5	43.0 ± 1.0	N/A
Iteration 4	35.2 ± 0.4	38.2 ± 0.8	43.8 ± 0.1	45.2 ± 1.2	N/A

Conclusion

- MCTS is used to augment LLMs with explicit search, enabling multi-step planning, better chain-of-thought reasoning, and complex decision-making. The LLM acts as a world model, policy, or value function guiding the tree search, similar to how AlphaZero combines policy/value networks with MCTS
- In training, MCTS helps generate high-quality reasoning traces and value estimates, which provide rich supervision for preference optimization or self-training
- At inference, MCTS helps select the best possible outputs by simulating future possibilities, balancing exploration and exploitation

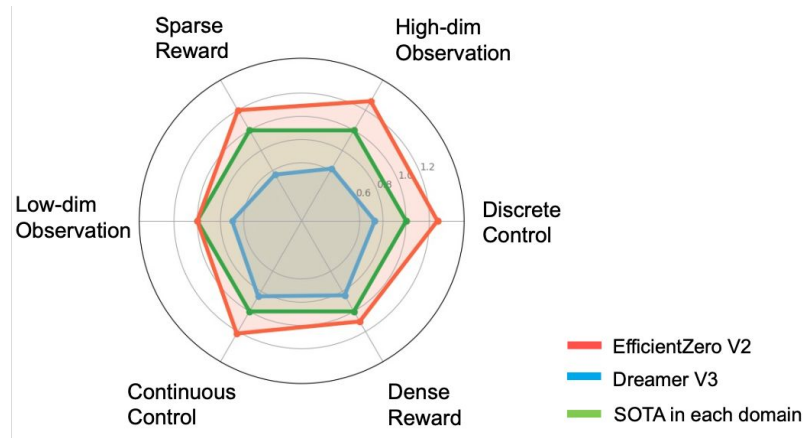




Extra

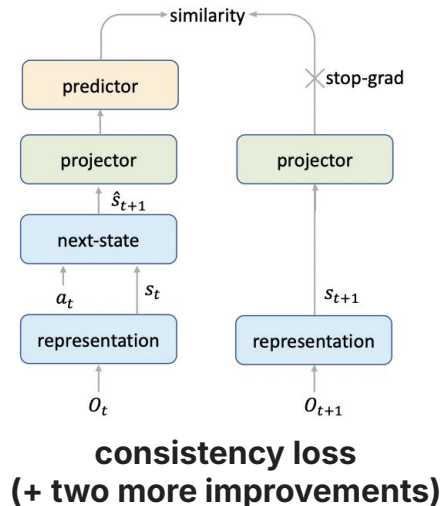
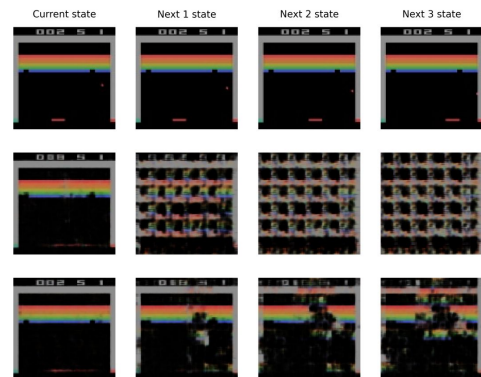
Open Source Implementations — EfficientZeroV2

- EfficientZeroV2 is a general framework designed for sample-efficient RL algorithms. Authors have expanded the performance of EfficientZero to multiple domains, encompassing both continuous and discrete actions, as well as visual and low-dimensional inputs.
- ICML 2024, Spotlight
- EfficientZero V2 outperforms the current state-of-the-art (SOTA) by a significant margin in diverse tasks under the limited data setting (including Dreamer-V3)



EfficientZero

Game	Full	w.o. consistency
Normed Mean	1.943	0.881
Normed Median	1.090	0.340

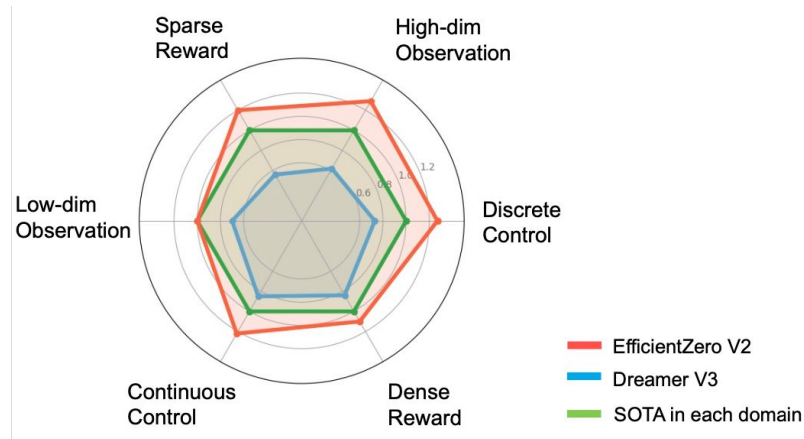


Game	Random	Human	MuZero	Ours
Alien	227.8	7127.7	530.0	808.5
Amidar	5.8	1719.5	38.8	148.6
Assault	222.4	742.0	500.1	1263.1
Asterix	210.0	8503.3	1734.0	2557.8
Bank Heist	14.2	753.1	192.5	351.0
BattleZone	2360.0	37187.5	7687.5	13871.2
Boxing	0.1	12.1	15.1	52.7
Breakout	1.7	30.5	48.0	414.1
ChopperCmd	811.0	7387.8	1350.0	1117.3
Crazy Climber	10780.5	35829.4	56937.0	83940.2
Demon Attack	152.1	1971.0	3527.0	13003.9
Freeway	0.0	29.6	21.8	21.8
Frostbite	65.2	4334.7	255.0	296.3
Gopher	257.6	2412.5	1256.0	3260.3
Hero	1027.0	30826.4	3095.0	9315.9
Jamesbond	29.0	302.8	87.5	517.0
Kangaroo	52.0	3035.0	62.5	724.1
Krull	1598.0	2665.5	4890.8	5663.3
Kung Fu Master	258.5	22736.3	18813.0	30944.8
Ms Pacman	307.3	6951.6	1265.6	1281.2
Pong	-20.7	14.6	-6.7	20.1
Private Eye	24.9	69571.3	56.3	96.7
Qbert	163.9	13455.0	3952.0	13781.9
Road Runner	11.5	7845.0	2500.0	17751.3
Seaquest	68.4	42054.7	208.0	1100.2
Up N Down	533.4	11693.2	2896.9	17264.2
Normed Mean	0.000	1.000	0.562	1.943
Normed Median	0.000	1.000	0.227	1.090

Mastering Atari Games with Limited Data

Open Source Implementations — EfficientZeroV2

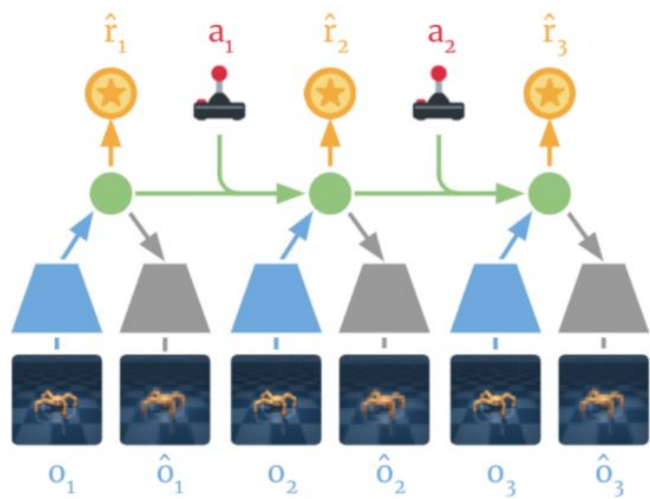
- EfficientZeroV2 is a general framework designed for sample-efficient RL algorithms. Authors have expanded the performance of EfficientZero to multiple domains, encompassing both continuous and discrete actions, as well as visual and low-dimensional inputs.
- ICML 2024, Spotlight
- EfficientZero V2 outperforms the current state-of-the-art (SOTA) by a significant margin in diverse tasks under the limited data setting (including Dreamer-V3)



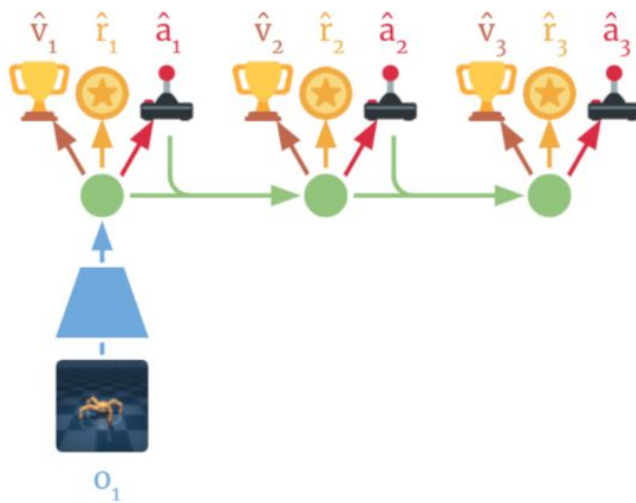
06

Подход Dreamer

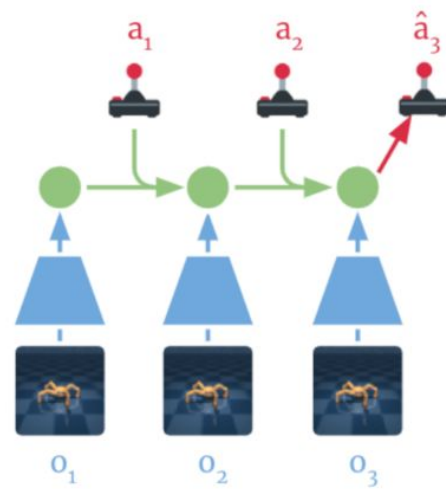
Dreamer



Learn dynamics from dataset



Learn behavior in imagination



Environment interaction

Псевдокод Dreamer-v1

Algorithm 1: Dreamer

Initialize dataset $\mathcal{D}$ with S random seed episodes.
Initialize neural network parameters θ, ϕ, ψ randomly.
while not converged do

for update step $c = 1..C$ do

 // Учим динамику среды

 Draw B data sequences $\{(a_t, o_t, r_t)\}_{t=k}^{k+L} \sim \mathcal{D}$.

 Compute model states $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$.

 Update θ using representation learning.

 // Учим поведение агента

 Imagine trajectories $\{(s_\tau, a_\tau)\}_{\tau=t}^{t+H}$ from each s_t .

 Predict rewards $E(q_\theta(r_\tau | s_\tau))$ and values $v_\psi(s_\tau)$.

 Compute value estimates $V_\lambda(s_\tau)$ via Equation 6.

 Update $\phi \leftarrow \phi + \alpha \nabla_\phi \sum_{\tau=t}^{t+H} V_\lambda(s_\tau)$.

 Update $\psi \leftarrow \psi - \alpha \nabla_\psi \sum_{\tau=t}^{t+H} \frac{1}{2} \|v_\psi(s_\tau) - V_\lambda(s_\tau)\|^2$.

 // Взаимодействие со средой

$o_1 \leftarrow \text{env.reset}()$

for time step $t = 1..T$ do

 Compute $s_t \sim p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$ from history. Восстанавливаем состояние по истории

 Compute $a_t \sim q_\phi(a_t | s_t)$ with the action model. Выбираем действие

 Add exploration noise to action.

$r_t, o_{t+1} \leftarrow \text{env.step}(a_t)$.

 Add experience to dataset $\mathcal{D} \leftarrow \mathcal{D} \cup \{(o_t, a_t, r_t)_{t=1}^T\}$.

Model components

Представление $p_\theta(s_t | s_{t-1}, a_{t-1}, o_t)$

Функция перехода $q_\theta(s_t | s_{t-1}, a_{t-1})$

Функция вознаграждения $q_\theta(r_t | s_t)$

Действия $q_\phi(a_t | s_t)$

Полезность $v_\psi(s_t)$

Hyper parameters

Seed episodes S

Collect interval C

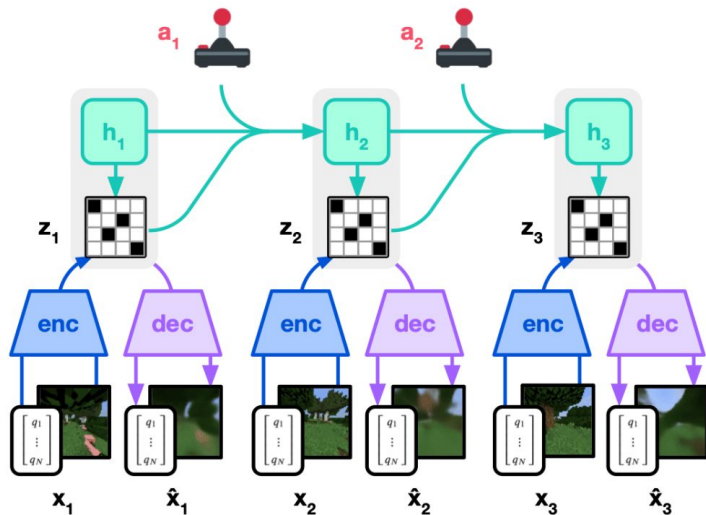
Batch size B

Sequence length L

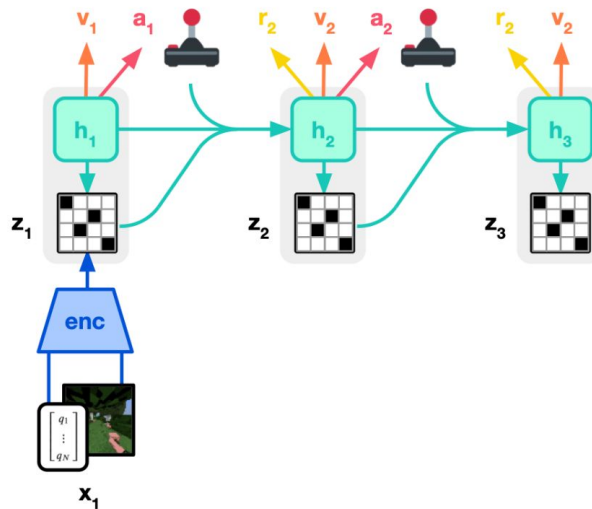
Imagination horizon H

Learning rate α

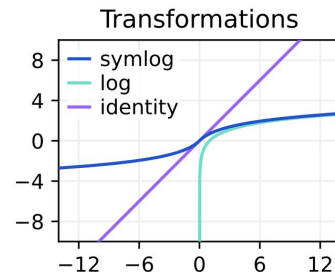
Dreamer-v3



(a) World Model Learning



(b) Actor Critic Learning

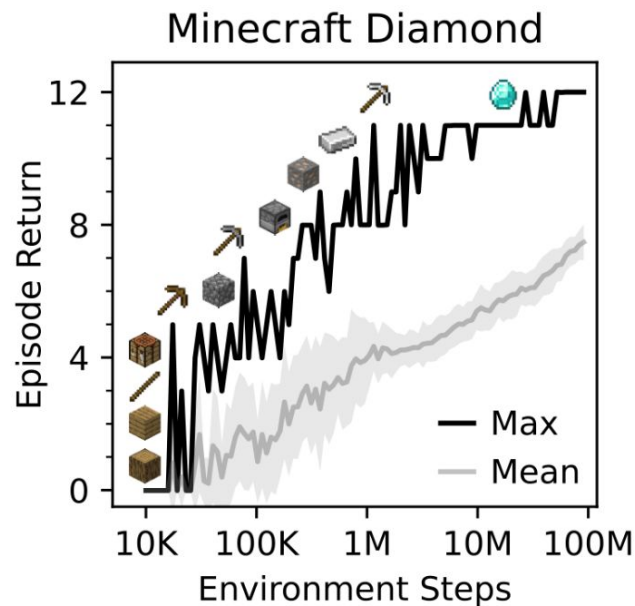
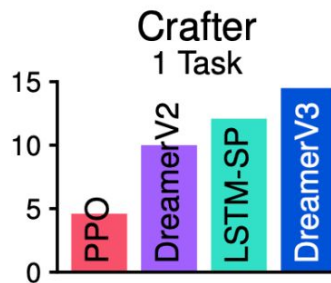
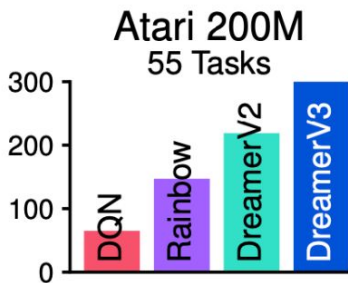
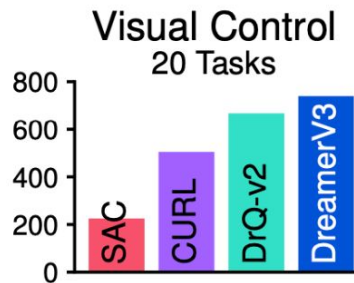
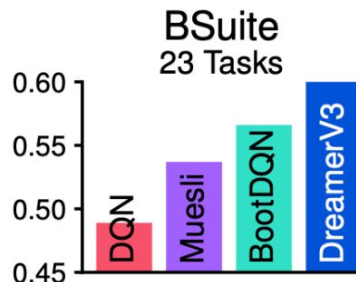
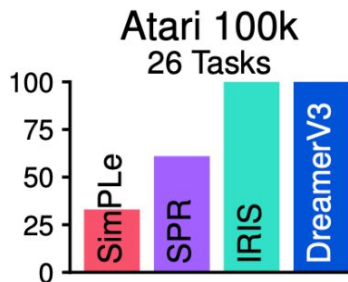
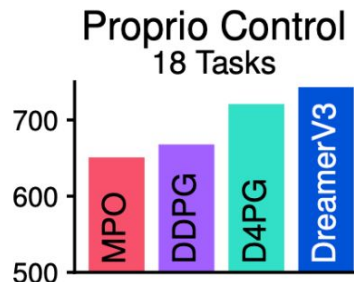


Symlog используется для того, чтобы решить проблему разного масштаба вознаграждений, от высоких и разреженных до низких и частых. – Способ нормализации.

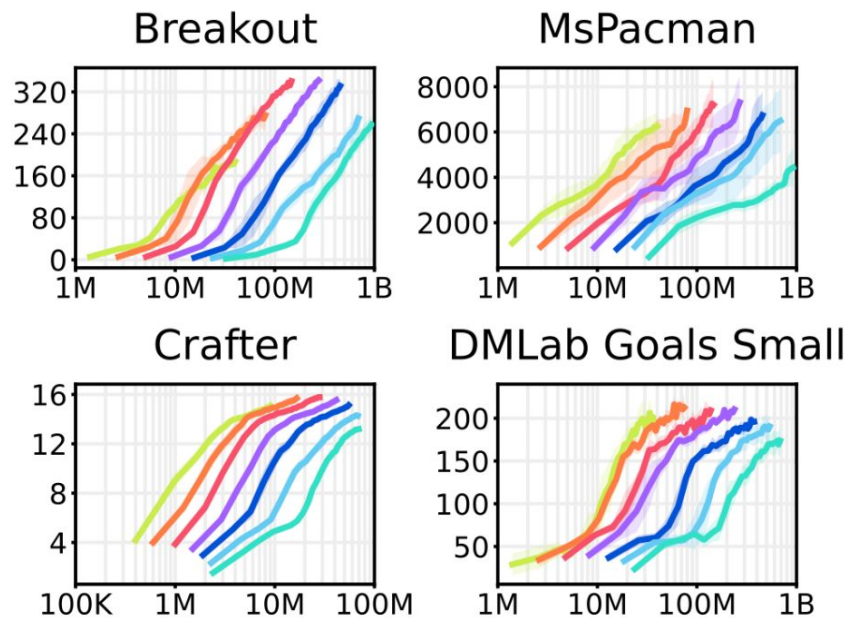
$$\text{symlog}(x) \doteq \text{sign}(x) \ln(|x| + 1)$$

Figure 3: Training process of DreamerV3. The world model encodes sensory inputs into a discrete representation z_t that is predicted by a sequence model with recurrent state h_t given actions a_t . The inputs are reconstructed as learning signal to shape the representations. The actor and critic learn from trajectories of abstract representations predicted by the world model.

Результаты

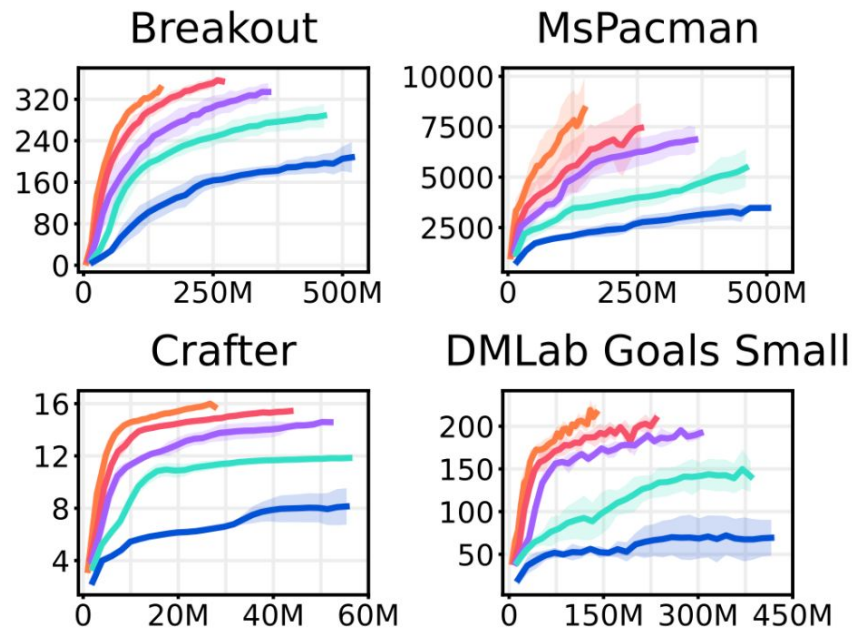


Увеличение модели



— 1 — 2 — 4 — 8 — 16 — 32 — 64

(a) Training Ratio



— XS — S — M — L — XL

(b) Model Size

Резюме:



- Сэмпл-эффективный алгоритм
- Низкая вычислительная эффективность (если смотрим не на шаги в среде, а на GPU часы)
- Dreamer-V3 можно применять почти для любой среды, без каких-либо доп. изменений
- Хороший выбор для медленных сред

Achievement	Rainbow	PPO	DreamerV2	APPO
Collect Coal	0.0%	0.4%	14.7%	96.6%
Collect Diamond	0.0%	0.0%	0.0%	22.6%
Collect Drink	24.0%	30.3%	80.0%	94.0%
Collect Iron	0.0%	0.0%	0.0%	83.1%
Collect Sapling	97.4%	66.7%	86.6%	98.9%
Collect Stone	0.2%	3.0%	42.7%	99.3%
Collect Wood	74.9%	83.0%	92.7%	99.9%
Defeat Skeleton	0.7%	0.2%	2.6%	89.3%
Defeat Zombie	39.6%	2.0%	53.1%	95.2%
Eat Cow	26.1%	12.0%	17.1%	93.3%
Eat Plant	0.0%	0.0%	0.1%	1.0%
Make Iron Pickaxe	0.0%	0.0%	0.0%	77.9%
Make Iron Sword	0.0%	0.0%	0.0%	55.4%
Make Stone Pickaxe	0.0%	0.0%	0.2%	97.6%
Make Stone Sword	0.0%	0.0%	0.3%	98.5%
Make Wood Pickaxe	4.8%	21.1%	59.6%	99.7%
Make Wood Sword	9.8%	20.1%	40.2%	99.0%
Place Furnace	0.0%	0.1%	1.8%	98.0%
Place Plant	94.2%	65.0%	84.4%	99.0%
Place Stone	0.0%	1.7%	29.0%	99.0%
Place Table	52.3%	66.1%	85.7%	99.0%
Wake Up	93.3%	92.5%	92.8%	97.0%
Score	4.3%	4.6%	10.0%	50.0%

Вопросы?